

Intelligent Selective Mechanism for Efficient Energy Harvesting from Hybrid Multimodal Energy Sources

Thesis to be submitted in fulfillment of the requirements for the degree

of

Bachelor of Technology Instrumentation Engineering

by

Roctotpal Sandilya

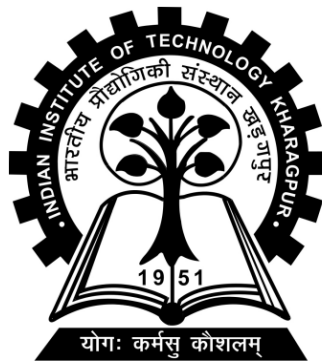
Roll No.: **22IE10035**

Under the guidance of

Dr. Banibrata Mukherjee

and

Dr. Somnath Sengupta



**ADVANCED TECHNOLOGY DEVELOPMENT CENTRE
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR**

MAY, 2025



Advanced Technology Development Centre
Indian Institute of Technology,
Kharagpur
India – 721302

CERTIFICATE

This is to certify that we have examined the thesis entitled “*Intelligent Selective Mechanism for Efficient Energy Harvesting from Hybrid Multimodal Energy Sources*”, submitted by **Roctotpal Sandilya** (Roll Number: **22IE10035**), a student of **Instrumentation Engineering, Indian Institute of Technology Kharagpur**, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology**.

We hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in fulfillment for the undergraduate degree for which it has been submitted. The thesis has fulfilled all the requirements as per the regulations of the Institute and has reached the standard needed for submission.

Dr. Banibrata Mukherjee
ATDC
IIT Kharagpur

Dr. Somnath Sengupta
ATDC
IIT Kharagpur

External Examiner

Place: Kharagpur
Date: 28-04-2025



Advanced Technology Development Centre
Indian Institute of Technology,
Kharagpur
India – 721302

DECLARATION

I hereby declare that this written submission represents my own work and ideas, carried out under the supervision of **Dr. Banibrata Mukherjee** and **Dr. Somnath Sengupta**, at the **Advanced Technology Development Centre, Indian Institute of Technology Kharagpur**. Wherever the work of others has been used or referred to, due acknowledgment has been made in the text.

I also declare that this thesis has not been submitted for the award of any other degree or diploma elsewhere.

Roctotpal Sandilya
Roll No.: **22IE10035**
Instrumentation Engineering
IIT Kharagpur
Date: 28-04-2025

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisors, **Dr. Banibrata Mukherjee** and **Dr. Somnath Sengupta**, for their constant guidance, encouragement, and valuable insights throughout the course of this project. Their support and constructive feedback have been instrumental in the successful completion of this work.

I am also thankful to the faculty members and staff of the **Advanced Technology Development Centre (ATDC), Indian Institute of Technology Kharagpur**, for providing the necessary facilities and a conducive environment for carrying out this project.

Roctotpal Sandilya

Roll No.: **22IE10035**

Instrumentation Engineering

IIT Kharagpur

ABSTRACT

The work presented in this thesis focuses on the design and development of an **intelligent selective mechanism for efficient energy harvesting from hybrid multimodal energy sources**. The initial system was designed for two sources; however, the concept has been **extended to a multi-source configuration** in this work. The motivation behind this extension is to enable efficient energy harvesting from multiple low-power sources operating under varying conditions of voltage and phase, thereby improving the overall conversion efficiency and reliability of the system.

As part of this extension, the **existing MSP430-based microcontroller code** was modified to support control and evaluation of **multiple input sources**. Corresponding **MATLAB and Simulink simulations** were carried out to validate the algorithmic performance and switching logic prior to hardware implementation. The system was designed to dynamically select the most suitable source or combination of sources based on real-time voltage and phase analysis, ensuring optimal energy transfer to the storage capacitor.

Given the **limited number of GPIO pins** available on the microcontroller, an efficient switching and measurement architecture was proposed using a combination of **shift registers and analog multiplexers (MUXs)**. This allowed scalable control of signal and blocking switches, along with sequential measurement of individual source voltages required for phase matching and selection. The proposed configuration enables the system to accommodate a greater number of input sources without additional hardware complexity.

The extended system architecture, simulation framework, and revised control algorithm together demonstrate a scalable and practical approach toward **multi-source intelligent energy harvesting**, suitable for applications in low-power embedded and self-sustaining systems.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Objectives of the Work	1
1.3	System Overview	2
1.4	Methodology	2
1.5	Organization of the Thesis	3
2	Literature Review	4
2.1	Introduction	4
2.2	Review of Energy Harvesting Sources and Techniques	4
2.3	Previous Work	6
2.4	Limitations of Earlier Work	7
2.5	Proposed Enhancements in the Present Work	7
2.6	Summary	9
3	Hardware Design	10
3.1	Functional Block Diagram	10
3.2	Circuit Diagram	11
3.3	Objectives and Design Constraints	11
3.4	System Block View	12
3.5	Per-Source Switching: SWS/SWB	12
3.5.1	Role of SWS and SWB	12
3.5.2	Switch Technology Choices	13
3.6	Shift-Register Chain (74HC595)	13
3.6.1	Purpose and Pin Economy	13
3.6.2	Bit Mapping and Timing	13
3.6.3	Output Polarity and Level	14
3.7	Analog MUX Bank (CD4051B, 3×8)	14
3.7.1	Topology and Addressing	14

4	Software Design and Implementation	15
4.1	Introduction of Software Design	15
4.2	High-Level Software Structure	15
4.3	Data Structures and Key Variables	17
4.4	Initialization / Setup	17
4.5	Core Functions — Description and Mapping	18
4.6	Evaluation Phase — Detailed Software Sequence	19
4.7	Primary Selection and Switch Configuration	20
4.8	Harvesting Phase — Real-Time Phase Matching	20
4.9	Timing, Settling and Safety Considerations	21
4.10	Pseudocode (One-page summary)	22
4.11	Summary	23
5	Simulation Modeling and Results	24
5.1	Introduction	24
5.2	Simulation Model Architecture	24
5.2.1	Energy Source Subsystem	25
5.2.2	Switching and Rectification Subsystem	26
5.2.3	Evaluation Capacitor Unit	27
5.2.4	Storage Capacitor and Power Conditioning	28
5.3	Control Algorithm Implementation	30
5.3.1	Evaluation Phase	30
5.3.2	Harvesting Phase	30
5.4	Governing Equations of Two-Source System	31
5.4.1	Case I: One Inductive and One Capacitive Source	31
5.4.2	Case II: Both Sources Inductive	32
5.5	Simulation Parameters	33
5.6	Results and Observations	33
5.7	Conclusion	34
6	Hardware setup and analysis	35
6.1	Experimental Setup	35
6.2	Experimental results	36
6.2.1	Intelligent Selective Mechanism	36
6.2.2	Conventional Configuration	38
6.2.3	Performance Improvement	39
6.3	Power Dissipation Analysis of Components	39
6.3.1	Microcontroller (MSP430G2553)	39
6.3.2	Switching Circuit (Optocouplers)	39

6.3.3	Signal Conditioning Circuits	39
6.3.4	Energy Storage and Regulation Circuits	40
6.4	Efficiency Calculation	40
7	Conclusion and Future Work	42
7.1	Conclusion	42
7.2	Future Work	43
8	Appendix	44

List of Figures

3.1	Block diagram of proposed system	10
3.2	Circuit diagram of proposed system	11
4.1	Flowchart of the firmware algorithm.	16
5.1	Simulink simulation of the system.	25
5.2	Input and control signal for 1 Hz.	26
5.3	Input and control signal for 20 Hz.	26
5.4	Input and control signal for 5 Hz.	26
5.5	Input and control signal for 10 Hz.	26
5.6	Evaluation capacitor voltage.	28
5.7	Evaluation capacitor voltage.	28
5.8	C-storage for 1 Hz.	29
5.9	C-storage for 10 Hz.	29
5.10	C-storage for 20 Hz.	29
5.11	C-storage for different voltages.	29
6.1	Experimental Setup.	35
6.2	Soldered Veroboard for 3 sources	36
6.3	Results fro 5Hz frequency.	36
6.4	Results fro 10Hz frequency.	36
6.5	Results fro 12Hz frequency.	37
6.6	Results fro 15Hz frequency.	37
6.7	Results fro 20Hz frequency.	37
6.8	Output voltage waveform of SEH at 14Hz frequency	38

List of Tables

5.1	Variation of Output Voltage with Frequency	34
6.1	Power Dissipation in Major Components	40

Chapter 1

Introduction

1.1 Background and Motivation

The increasing demand for self-powered embedded systems and low-power electronics has brought significant attention to the domain of **energy harvesting**. Energy harvesters aim to extract and convert ambient energy from multiple sources such as solar, thermal, vibration, and electromagnetic fields into usable electrical energy. Such systems are especially relevant for remote sensors, biomedical devices, and distributed monitoring nodes, where replacing or recharging batteries is either impractical or inefficient.

Traditionally, energy harvesters are designed to operate with a single energy source, or in some cases, with two sources using simple selection or priority-based switching mechanisms. However, ambient energy availability is highly dynamic, and the optimal source for energy extraction can vary over time. To address this limitation, the present work focuses on developing an **intelligent multi-source selection and harvesting mechanism** capable of evaluating, prioritizing, and combining multiple sources in real time.

1.2 Objectives of the Work

The primary objective of this work is to extend the concept of a dual-source harvester to a **scalable multi-source energy harvesting system**. The project aims to implement a selection algorithm that can dynamically identify the most suitable energy source(s) based on instantaneous voltage and phase relationships. The key goals of this thesis are:

- To design and implement a programmable architecture that can accommodate **multiple input sources** for hybrid energy harvesting.
- To modify and extend the existing **MSP430-based controller code** to handle

the evaluation and selection logic for N sources.

- To perform **MATLAB/Simulink simulations** to validate system functionality, selection timing, and capacitor voltage dynamics.
- To propose a scalable hardware interface using **shift registers and analog multiplexers (MUXs)** for efficient control of switching and voltage measurement under limited GPIO availability.
- To experimentally verify the operation with two and three sources before extending it to higher configurations.

1.3 System Overview

The developed system consists of multiple input sources connected to an evaluation and storage capacitor network, controlled by a low-power microcontroller (MSP430). Each source is associated with signal and blocking switches that determine its connection state during the *evaluation* or *harvesting* phases. During evaluation, the controller sequentially connects each source to the evaluation capacitor, measures the corresponding voltage rise, and determines the most effective source based on ΔV_{cap} and phase alignment.

Once the optimal source (or combination of sources) is identified, it is connected to the storage capacitor for sustained energy transfer. The system continuously monitors instantaneous source voltages to ensure phase matching and maintain efficient energy flow.

To overcome the hardware limitations posed by the finite number of MCU pins, the proposed architecture integrates a **Serial-In Parallel-Out (SIPO) shift register** for driving multiple switches and a **3×8 analog multiplexer** for sequentially measuring source voltages. This approach enables the design to scale to 8, 16, or more sources without increasing the microcontroller’s pin count, while maintaining precise timing and low power consumption.

1.4 Methodology

The development process involved both software and hardware stages:

1. **Algorithm Development:** The dual-source MSP430 algorithm was generalized for N sources, incorporating evaluation sequencing, real-time phase detection, and adaptive switching control.

2. **Simulation:** MATLAB/Simulink models were constructed to simulate the charging and discharging dynamics of the evaluation and storage capacitors, as well as the selection behavior under multiple source conditions.
3. **Hardware Implementation:** The proposed multi-source interface circuit was designed using MUXs and shift registers, enabling digital control of signal and blocking switches with minimal pin usage.
4. **Testing and Validation:** Experimental verification was carried out on a bread-board prototype using two and three sources. The system was observed to perform accurate source selection and efficient charge transfer between evaluation and storage capacitors.

1.5 Organization of the Thesis

The rest of this thesis is organized as follows:

- **Chapter 2** presents the literature review and theoretical background of energy harvesting techniques, power management circuits, and previous multi-source approaches.
- **Chapter 3 and 4** details the system design, including the hardware configuration, control flow, and the proposed multiplexing and shift-register-based switching architecture.
- **Chapter 5** discusses the MATLAB/Simulink modeling and simulation results for multi-source evaluation and harvesting.
- **Chapter 6** provides the hardware implementation details, experimental setup, and performance evaluation.
- **Chapter 7** concludes the thesis and suggests directions for future work.

Through this structured development and testing process, the proposed system successfully demonstrates a scalable and efficient approach for **intelligent multi-source energy harvesting**, paving the way for self-sustaining embedded applications.

Chapter 2

Literature Review

2.1 Introduction

Energy harvesting has emerged as a promising technology for powering low-power electronic systems by converting ambient environmental energy into electrical energy. The concept enables the deployment of self-sustaining sensor nodes, wearables, and remote electronic devices without reliance on conventional batteries. This chapter presents a review of various energy harvesting sources and techniques, followed by a discussion of existing architectures and their limitations. The chapter concludes with the motivation and approach adopted in this work to overcome those limitations.

2.2 Review of Energy Harvesting Sources and Techniques

A wide range of ambient energy sources can be exploited for micro-scale power generation. The most common categories include solar (photovoltaic), thermal (thermoelectric), vibrational (piezoelectric and electromagnetic), and radio frequency (RF) energy harvesting. Each source exhibits unique energy conversion characteristics, power density, and circuit-level integration challenges.

2.2.1 Solar Energy Harvesting

Photovoltaic (PV) cells are among the most mature and widely used energy-harvesting devices. They convert incident light into electrical energy using semiconductor junctions, offering power levels ranging from a few milliwatts to several hundred milliwatts depending on illumination conditions [1], [2]. This makes small-scale PV modules particularly suitable for both indoor and outdoor sensor systems. However, their practical

operation is affected by variations in irradiance, temperature sensitivity, and the requirement for maximum-power-point tracking (MPPT) to ensure optimal efficiency. Several prior works, including device-level studies and low-power circuit implementations from Texas Instruments and Analog Devices, have demonstrated ultra-low-power MPPT and power-conditioning circuits tailored for energy-harvesting applications [5], [9].

2.2.2 Thermal Energy Harvesting

Thermoelectric generators (TEGs) utilize the Seebeck effect to generate a voltage from temperature differences across two dissimilar materials, making them well-suited for industrial environments and body-worn electronics where small but continuous thermal gradients exist [1], [2]. Although TEGs offer high reliability and no moving parts, their electrical output is typically very low—often only a few tens of millivolts—necessitating the use of ultra-efficient boost converters for practical energy extraction. Prior work and device-level demonstrations from Analog Devices have highlighted specialized ICs such as the LTC3108 and the LTC3388 series that are optimized for low-voltage, energy-harvesting scenarios [5]. In particular, the LTC3388 IC used in this project provides high efficiency and extremely low quiescent current, making it especially suitable for intermittent, low-input-voltage harvesting applications encountered in thermoelectric systems.

2.2.3 Vibration-Based Energy Harvesting

Vibrational energy can be harvested using piezoelectric, electromagnetic, or electrostatic mechanisms. Piezoelectric vibration harvesting, widely studied by Mitcheson et al. [1] and Tadesse et al. [3], exploits the ability of materials such as PZT and PVDF to generate charge when mechanically strained. These materials provide high energy density near resonance and are widely used in compact sensor nodes.

Early work by Paradiso and Starner [2] highlighted the potential of electromagnetic harvesters, where a moving magnet-coil structure induces voltage through Faraday’s law. Although effective for larger vibration amplitudes, these devices require greater mechanical displacement and hence have larger form factors.

Electrostatic (variable-capacitance) harvesters, also discussed in the broader vibration-harvesting literature [1], use a changing capacitance structure to convert mechanical motion into electrical output. While they offer advantages in microfabricated implementations, they require an external bias source and careful control circuitry. Overall, these mechanisms demonstrate that vibrational energy is highly dependent on frequency content and mechanical excitation levels, and different transduction types are suited to different deployment environments.

2.2.4 Radio Frequency (RF) and Magnetic Induction Energy Harvesting

RF energy harvesting converts ambient electromagnetic radiation—such as Wi-Fi, GSM, LTE, and RFID signals—into usable DC power through a rectifying antenna (rectenna) structure. Foundational work by Mitcheson et al. [1] and Paradiso and Starner [2] highlighted the feasibility of scavenging microwatt-level RF energy in urban and indoor environments. Although the available power density is typically low compared to mechanical or solar sources, RF harvesting remains attractive for low-power wireless sensor networks, especially where communication infrastructure already exists. Magnetic-induction-based wireless power transfer, also widely covered in contemporary energy-harvesting literature [1], offers a complementary near-field approach in which power is coupled through resonant coils. This method enables higher transfer efficiency at short distances and is increasingly explored for IoT devices, wearable electronics, and biomedical implants.

2.2.5 Hybrid and Multi-Source Energy Harvesting

Hybrid or multisource energy harvesting architectures have been proposed to overcome the inherent intermittency of individual sources. Early studies by Mitcheson et al. [1] and Paradiso and Starner [2] emphasized that no single modality—solar, vibration, thermal, or RF—provides stable output across all environmental conditions, motivating the use of combined harvesters. Prior implementations typically rely on diode-OR networks, passive power combiners, or simple logic-controlled switches to merge sources, as seen in commercial application notes from Texas Instruments [6] and Analog Devices [7]. However, these conventional approaches do not account for phase coherence among AC-type sources and often introduce reverse current leakage or inefficient rectification, which significantly limits overall system efficiency. This limitation forms the core motivation for the dynamically controlled multisource architecture proposed in this work.

2.3 Previous Work

A significant portion of the foundational work for this thesis was carried out by Aman Raj[10] during his Master’s Thesis Project at IIT Kharagpur. His research focused on developing an intelligent dual-source hybrid energy harvesting system capable of selectively utilizing the most suitable source based on instantaneous operating conditions. His implementation used two AC energy sources, each connected through a pair of switching elements—a signal switch (SWS) and a blocking switch (SWB)—controlled by an MSP430 microcontroller.

To determine the superior source at any given moment, Aman Raj implemented an evaluation capacitor-based selection mechanism, wherein each source was sequentially connected to a small capacitor. The microcontroller measured the resulting voltage rise to estimate the instantaneous effective power contribution of each source. The source yielding the highest voltage rise was identified as the primary source, while the second source was dynamically added or removed based on a real-time phase comparison algorithm, ensuring constructive addition and avoiding destructive interference. His work successfully demonstrated an intelligent control mechanism that improved harvesting efficiency compared to passive OR-based or statically switched designs. While the design was novel and functionally validated, the dual-source architecture posed multiple limitations when considering system scalability.

2.4 Limitations of Earlier Work

Earlier implementations of hybrid energy harvesting systems at the microcontroller level have primarily focused on two sources. The existing dual-source design used a pair of signal and blocking switches for each source and evaluated the voltage difference across an evaluation capacitor to identify the stronger source. While this approach proved functional for small-scale configurations, it faced several constraints when extended beyond two sources:

1. **Limited scalability:** Each additional source required two dedicated GPIO pins for control and one analog input pin for measurement, leading to rapid exhaustion of the available microcontroller I/O.
2. **Hardware complexity:** Adding more sources proportionally increased wiring density and signal routing complexity on the board.
3. **Algorithmic rigidity:** The existing firmware was hard-coded for two sources, with a fixed evaluation-harvest sequence. Extending it to more sources required structural changes in both code and timing logic.

As a result, scaling the system to more than two sources without redesigning the control and measurement scheme was impractical. These limitations motivated the need for a more generalized, modular, and resource-efficient approach.

2.5 Proposed Enhancements in the Present Work

To overcome the aforementioned challenges, the present work introduces several key architectural and algorithmic modifications:

2.4.1 Scalable Multi-Source Control Using Shift Registers

Given the limited number of digital output pins available on the MSP430 microcontroller, a serial-to-parallel interface was implemented using a **Serial-In Parallel-Out (SIPO) shift register**. This allows a single data line, clock, and latch signal to control multiple switch gates. Each output of the shift register corresponds to the enable line of a signal or blocking switch. Thus, with one 8-bit shift register, up to eight switches can be controlled using only three MCU pins. Additional registers can be cascaded to support higher source counts without modifying the core circuit.

2.4.2 Sequential Measurement Using Analog Multiplexer

For voltage measurement of each source, a **3×8 analog multiplexer (MUX)** was introduced, allowing the ADC to sample multiple analog inputs sequentially through a single channel. This significantly reduces the number of ADC pins required, while maintaining accurate voltage and phase readings. The multiplexer selection lines are driven by the microcontroller, synchronized with the shift-register outputs to ensure only one source is measured at a time.

2.4.3 Algorithmic Extension for N Sources

The selection algorithm was generalized to handle an arbitrary number of sources. The evaluation phase was restructured into a looped sequence controlled by an index counter, allowing each source to be evaluated one by one. For every cycle:

1. The corresponding switches are activated through the shift register.
2. The MUX selects the respective source for voltage measurement.
3. The microcontroller computes the ΔV_{cap} for that source.

At the end of the evaluation phase, the source (or combination) yielding the highest energy transfer potential is connected to the storage capacitor for harvesting.

2.4.4 Simulation and Verification

The proposed multi-source architecture was modeled and validated in **MATLAB/Simulink**. The simulation captured real-time capacitor voltage dynamics and source selection behavior under varying voltage and phase conditions. This helped verify both control logic and hardware feasibility before physical implementation.

2.4.5 Component Selection and Documentation

The hardware design and analysis were supported by datasheets and application notes from key components, including:

- **LTC3388:** Ultra-low quiescent current DC-DC converter for efficient voltage regulation from low-level sources.
- **AQV202 and TLP222A:** PhotoMOS relays used for solid-state switching of signal and blocking paths.
- **74HC595:** 8-bit SIPO shift register used for expanding GPIO control lines.
- **CD4051B:** 3×8 analog multiplexer used for sequential source voltage measurement.

These components were chosen for their low power consumption, high reliability, and compatibility with low-voltage microcontroller systems.

2.6 Summary

This chapter presented an overview of various energy harvesting sources, reviewed existing hybrid harvester designs, and discussed the limitations of earlier two-source architectures. The proposed multi-source configuration introduces a scalable control structure using shift registers and multiplexers, along with an adaptive algorithm capable of efficient energy selection and management. The following chapter details the system design and implementation methodology in both hardware and simulation environments.

Chapter 3

Hardware Design

3.1 Functional Block Diagram

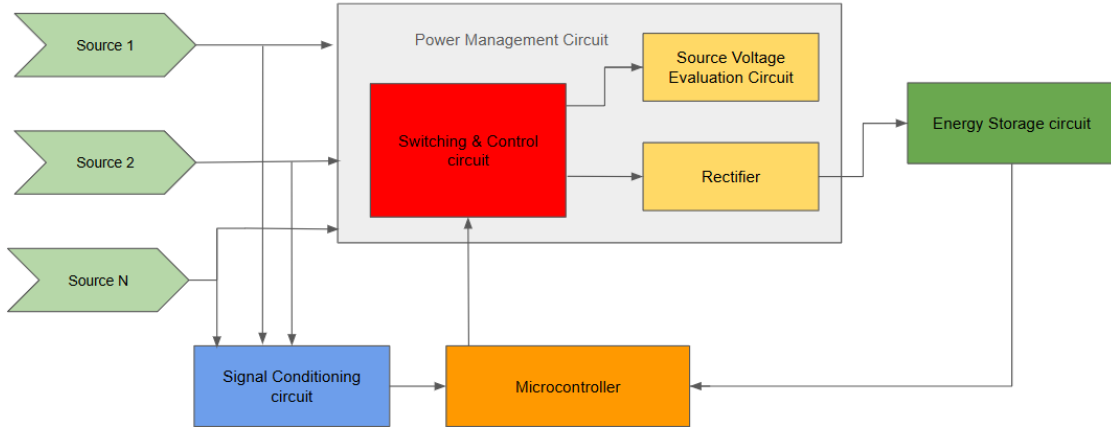


Figure 3.1: Block diagram of proposed system

The overall architecture of the proposed multi-source energy harvesting system is illustrated in Fig. 4.1. A set of heterogeneous energy harvesting sources, denoted as Source 1 to Source N, form the primary inputs of the system. Each source is interfaced through a Signal Conditioning Circuit, which ensures level shifting, filtering, and impedance matching so that the raw transducer outputs are compatible with the analog front-end of the microcontroller and the power management circuitry. The conditioned signals are forwarded to the Microcontroller, which serves as the central supervisory unit responsible for source monitoring, MUX addressing, switch control via the shift-register chain, and execution of the evaluation–harvesting algorithm. Based on the sensed voltages and the programmed control logic, the microcontroller issues appropriate commands

to the Switching and Control Circuit, a part of the Power Management Circuit, which dynamically connects or isolates individual sources during evaluation and harvesting. The power management block also includes a Source Voltage Evaluation Circuit, used to assess each source's instantaneous capability by temporarily charging an evaluation capacitor, and a Rectifier, which ensures unidirectional energy flow toward the storage subsystem. The processed and rectified output is finally delivered to the Energy Storage Circuit, comprising the storage capacitor and associated regulation elements, where the accumulated energy is stored and made available for downstream loads. This modular and scalable architecture enables seamless integration of multiple ambient sources while providing adaptive control for maximizing the net harvested energy.

3.2 Circuit Diagram

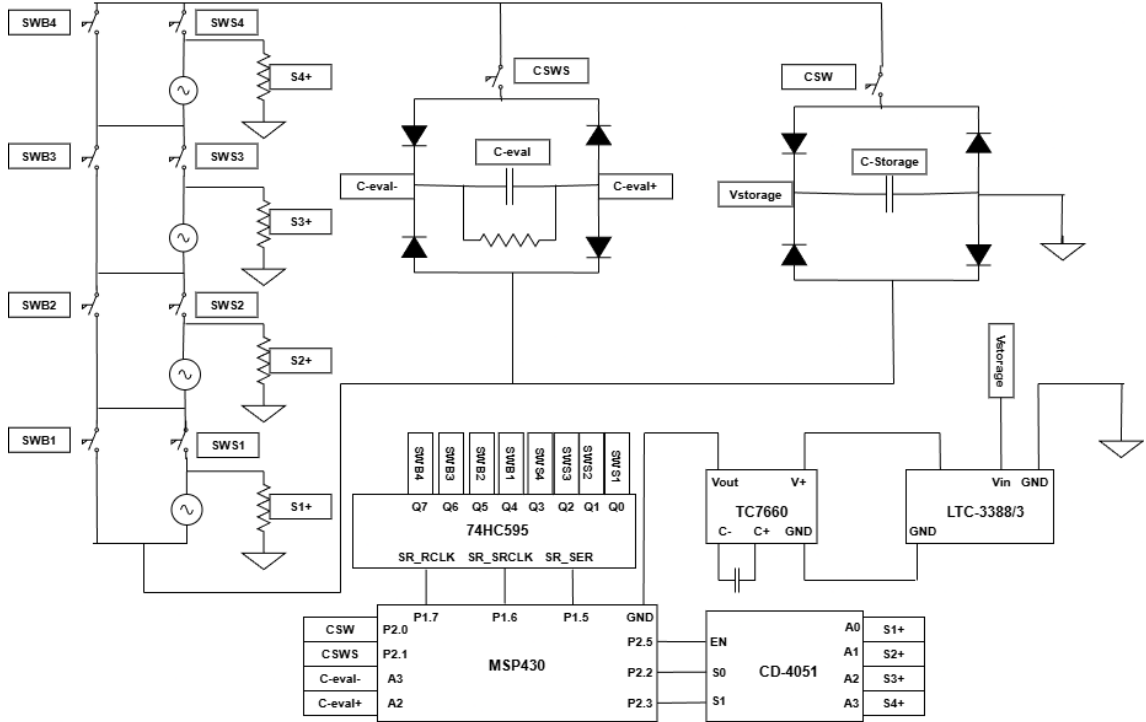


Figure 3.2: Circuit diagram of proposed system

3.3 Objectives and Design Constraints

The hardware architecture shown in Fig. 3.2 is designed to satisfy four key functional requirements: (i) interface up to N input energy sources (with a target scalability of 16–24) while using a minimal number of microcontroller I/O pins, (ii) evaluate each source under identical electrical conditions using a common evaluation capacitor to ensure unbiased

comparison, (iii) harvest energy from the strongest source while selectively enabling additional sources that satisfy the phase-alignment condition, and (iv) maintain a low-leakage and electrically robust structure suitable for low-power energy harvesting applications.

As illustrated in Fig. 3.2, the design is subject to multiple constraints, including the limited number of available GPIOs on the MSP430 microcontroller, low signal amplitudes (typically below 3.3 V), and the necessity to prevent unintended cross-coupling between multiple connected sources. These considerations motivated the adoption of a multiplexed voltage-sensing framework, controlled switching topology, and careful signal routing to minimize parasitic effects and reverse current flow, thereby ensuring stable and reliable operation in a scalable multi-source harvesting environment.

3.4 System Block View

A high-level representation of the proposed architecture is shown in Fig. 3.1. The system is designed to interface with multiple ambient energy sources and intelligently route, measure, and harvest energy based on real-time operating conditions. The major functional blocks are summarized below:

- A bank of ambient sources $S_0 \dots S_{N-1}$ (solar/TEG/piezo DC outputs).
- Per-source switching: **SWS** (signal switch) and **SWB** (block switch).
- A **74HC595 SIPO shift-register** chain that drives all SWS/SWB gates.
- A **3×8 analog MUX bank** (CD4051B) that routes any source to a single ADC pin.
- An **evaluation capacitor** C_{eval} and a **storage capacitor** C_{stor} with MCU-controlled switches CSW/CSWS.
- An ultra-low-I_q **LTC3388** DC–DC stage for regulated output and system self-powering.
- Reference, protection, and measurement points for bring-up.

3.5 Per-Source Switching: SWS/SWB

3.5.1 Role of SWS and SWB

For each source i , two controlled paths exist:

- **SWS_i** connects the source into the bus (evaluation or storage path).
- **SWB_i** actively *blocks* or isolates the source to prevent back-feeding/cross-coupling.

The firmware maps bit $2i$ to SWS_i and bit $2i + 1$ to SWB_i inside a 64-bit `shiftState`.

3.5.2 Switch Technology Choices

Three options are common:

- **PhotoMOS (e.g., AQV202/TLP222A):** Near-ideal isolation, low leakage, $m\Omega$ – Ω on-resistance, μA coil drive via LED side. Best for accuracy and isolation; slightly higher BOM cost.
- **Small MOSFETs:** SOT-23 N/P-FETs in back-to-back topology for bi-directional isolation. Lowest on-resistance but needs careful gate drive and body-diode management.
- **Analog switch ICs:** (e.g., ADG14xx) integrate many channels but share substrate—beware feedthrough; best used on measurement/MUX side.

For repeatable evaluation, PhotoMOS is simplest and robust; storage path may use FETs for lowest loss.

3.6 Shift-Register Chain (74HC595)

3.6.1 Purpose and Pin Economy

Controlling $2N$ switch gates directly would exhaust MCU pins. A daisy-chain of 74HC595 devices exposes $8M$ parallel outputs from only three MCU pins: SER (P1.5), SRCLK (P1.6), RCLK (P1.7).

3.6.2 Bit Mapping and Timing

- **Bit map:** $\text{bit}(2i) = \text{SWS}_i$, $\text{bit}(2i + 1) = \text{SWB}_i$.
- **Update:** Firmware shifts MSB-first for `shiftBits` cycles, then latches. Gate transitions are thus synchronous—no intermediate illegal states.
- **Decoupling:** Place 100 nF at each 74HC595 VCC, and a bulk 4.7–10 μF per chain.

Gate-driver current for PhotoMOS LEDs is low; series resistors (1–4.7 k Ω) set LED current to ~ 1 –3 mA per active channel.

3.6.3 Output Polarity and Level

If external switch logic is active-low, invert bit conventions in firmware or add inverters. Keep 74HC595 on the 3.3V domain (same as MSP430) to avoid level shifters.

3.7 Analog MUX Bank (CD4051B, 3×8)

3.7.1 Topology and Addressing

Three CD4051B parts provide up to 24 channels. Common OUT pins are connected to the ADC input A0

Chapter 4

Software Design and Implementation

4.1 Introduction of Software Design

This chapter describes the embedded software developed for the MSP430 microcontroller that controls the multi-source energy harvesting system. The firmware implements two main operational phases: (1) an *evaluation* phase that sequentially measures each source’s ability to charge an evaluation capacitor, and (2) a *harvesting* phase in which the best (primary) source is connected to the storage capacitor while in-phase secondary sources are added dynamically. The high-level flow of the algorithm is summarized in the flowchart shown in Figure 4.1.

The firmware is written in C for the MSP430 (ADC10 module, Timer_A, and GPIO), and interfaces to two external hardware primitives: (1) a daisy-chained 74HC595 SIPO shift register chain for driving all SWS/SWB outputs, and (2) a bank of $3 \times$ CD4051 analog multiplexers that route up to 24 source voltages into the ADC. The following sections break the software into functional parts, explain timing and control requirements, map code functions to algorithmic steps, and provide implementation and validation advice.

4.2 High-Level Software Structure

The overall firmware operation is summarized in the flowchart shown in Fig. 4.1. The software executes continuously inside an infinite loop and is organized into three primary phases, each corresponding to a distinct functional role in the harvesting process:

1. **Initialization and System Setup** — During startup, the microcontroller configures the system clock, I/O pins, ADC modules, Timer_A, and communication lines for the shift-register and multiplexer interface. At the end of initialization, all input sources are forced into a blocked state by updating the shift register, ensuring the system begins in a known and electrically safe configuration.

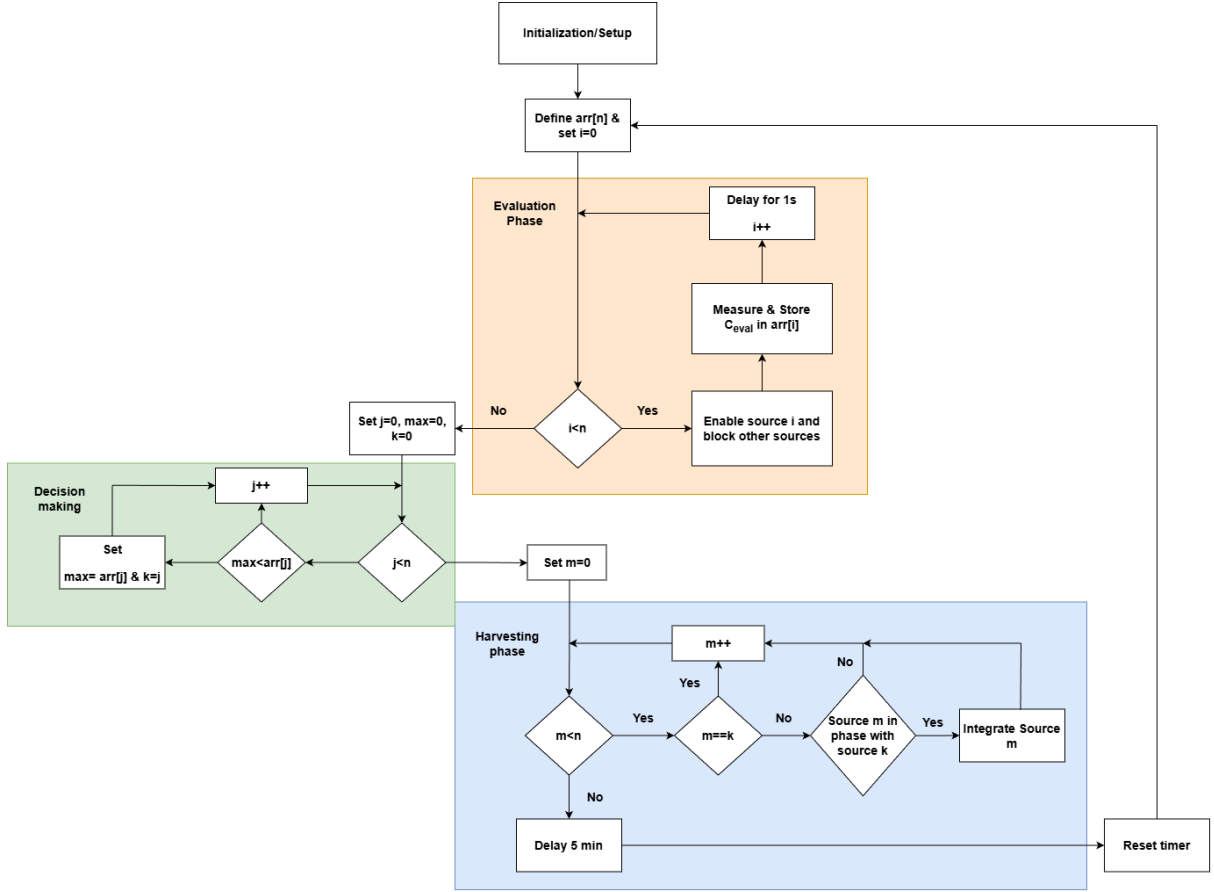


Figure 4.1: Flowchart of the firmware algorithm.

2. **Evaluation Phase** — In this phase, each source $i = 0 \dots N - 1$ is evaluated sequentially:

- All sources are first blocked, after which only source i is enabled through the controlled switching network.
- The evaluation capacitor C_{eval} is momentarily connected using CSW, allowing it to charge for a fixed interval. The voltages at the high and low capacitor nodes are then sampled using the ADC to compute the differential response ΔV_i , representing the instantaneous energy potential of source i .
- The capacitor is disconnected and the source is blocked again, followed by a controlled delay to allow stabilization and avoid measurement interference before the next iteration.

All computed values ΔV_i are stored in the array `evalDiffs[]` for later comparison.

3. **Selection and Harvesting Phase** — Once all sources have been evaluated, the source with the maximum ΔV is selected as the primary harvester. The storage capacitor C_{stor} is then connected via CSWS, and the system enters an adaptive harvesting loop where:

- (a) A reference voltage (A4) is sampled to form a measurement baseline.
- (b) The voltage of the primary source is read through the analog multiplexer.
- (c) Each remaining source is sampled in succession, and its instantaneous value is compared to that of the primary. If the polarity matches and the contribution is in-phase, the source is enabled; otherwise, it is blocked to prevent destructive interference.

This harvesting loop continues for a predefined time window, after which timers are reset and the process returns to the evaluation phase.

This modular structure enables scalable control logic suitable for large numbers of heterogeneous input sources, while maintaining deterministic timing and minimal computation overhead.

4.3 Data Structures and Key Variables

Relevant data structures and global variables used in firmware:

- `#define MAX_SOURCES` — compile-time number of sources (≤ 24).
- `int evalDiffs[MAX_SOURCES]` — stores computed ΔV for each source (signed).
- `unsigned long long shiftState` 64-bit bitfield representing SWS/SWB outputs for all sources (bit pair per source: $\text{bit}(2*i)=\text{SWS}_i$, $\text{bit}(2*i+1)=\text{SWB}_i$).
- `int shiftBits` — number of bits to shift ($2 * \text{MAXSOURCES}$).
- ADC result buffer: `volatile unsigned int adc_results[8]` — indexed storage for ISR results (used as scratch).
- Timing variables: `volatile unsigned long overflowTicks` for Timer_A overflows, and float `totalTime` plus `startTime` and `endTime`.

4.4 Initialization / Setup

All hardware resources are configured in the `setup()` function:

1. Stop watchdog and configure DCO to 1 MHz for predictable delays.
2. Configure GPIO directions:
 - ADC pins (P1.0–P1.4) as inputs (ADC10).

- Shift register control pins P1.5 (SER), P1.6 (SRCLK), P1.7 (RCLK) as outputs.
 - MUX address pins P2.2–P2.4 and bank enables P2.5–P2.7 as outputs.
 - Capacitor control switches CSW (P2.0) and CSWS (P2.1) as outputs.
3. Initialize ADC10 (reference $V_{cc}-V_{ss}$, S/H time, enable interrupts).
 4. Configure Timer_A to measure long durations (SMCLK with divider and overflow counting).
 5. Set all MUX banks disabled (EN HIGH) and CSW/CSWS LOW.
 6. Initialize `shiftState` to block all sources via `blockAllSources()`, which writes SWS=0 and SWB=1 for every source and latches via `updateShiftRegister()`.

Design rationale: blocking all sources on power up ensures no unintentional currents or shorts during bring-up. Running the MCU at 1 MHz provides the timing constants used through the code (`'delaycycles()'`), but the software uses explicit delays conservatively; these can be tuned.

4.5 Core Functions — Description and Mapping

This section maps the primary C functions to their algorithmic intentions and provides implementation notes.

`analogReadToIndex(pin, targetIndex)`

- Purpose: Configure ADC10 to read channel ‘pin’ and store the result in `adc_results[targetIndex]` via the ISR.
- Operation: Disable ENC, set `ADC10CTL1` to select channel, enable analog input bit in `ADC10AEO`, set pin as input, then start conversion with `ENC + ADC10SC`.

`readMuxedSource(sourceIndex)`

- Purpose: Read an arbitrary source voltage routed through one of the three CD4051 MUX banks using a single ADC input (P1.0).
- Operation: Disable all MUX banks, set address pins S0..S2 for the target channel, enable the appropriate bank (active LOW assumed), wait for MUX settling, call `analogReadToIndex(ADC_SOURCE_CH, 0)` and wait for ISR, then disable the bank and return `adc_results[0]`.

updateShiftRegister()

- Purpose: Serially shift the current `shiftState` bitfield into the daisy-chained 74HC595 devices and latch outputs simultaneously.
- Operation: Drive RCLK low, shift `shiftBits` bits MSB-first by setting SER and pulsing SRCLK, then latch by pulsing RCLK. Small delays (few CPU cycles) are used for setup/hold.

setSourceSwitches(sourceIndex, swson, swbon)

- Purpose: Toggle the SWS/SWB pair for a given source in the `shiftState` bitfield and update the register.
- Operation: Set or clear the two bits corresponding to that source and call `updateShiftRegister()`.
- Notes: Use this function to atomically set the desired state for any single source. For multi-source changes (e.g., block all then enable primary), it is better to build the entire `shiftState` and then call `updateShiftRegister()` once to avoid intermediate invalid states.

blockAllSources()

- Purpose: Convenience function to set all SWS=0 (off) and SWB=1 (block) across all sources and update shift register.
- Implementation: Loops through each source bit pair and adjusts `shiftState` accordingly before calling `updateShiftRegister()`.

4.6 Evaluation Phase — Detailed Software Sequence

The evaluation phase is the part of the firmware that ranks all sources. The sequence implemented is:

1. For each source index `i = 0..MAX_SOURCES-1`:
 - (a) Call `blockAllSources()` to ensure all are blocked.
 - (b) Call `setSourceSwitches(i, true, false)` to enable only source `i`.
 - (c) Close the evaluation capacitor switch: `P2OUT = CSW`—.
 - (d) Delay for charging: `delaycycles(500000)` (=0.5 s).
 - (e) Measure capacitor high (A2) and low (A3) using `analogReadToIndex()`; compute `evalDiffs[i] = ch - cl`.

- (f) Open CSW: `P2OUT &= ~CSW`.
- (g) Block the evaluated source: `setSourceSwitches(i, false, true)`.
- (h) Delay for discharge/stabilization: `delaycycles(1000000)` (=1.0 s).

Implementation notes:

- The fixed charging/dischage delays are conservative. In later iterations these can be reduced if the capacitor, source internal resistance, and measurement noise permit.
- To make the evaluation faster, consider measuring capacitor voltage continuously during the 0.5 s window and sampling the final value only at the end (or reducing the charge window based on expected source impedance).
- All evaluations use the same capacitor as a uniform load ensuring fair comparison.

4.7 Primary Selection and Switch Configuration

After collecting `evalDiffs[i]` for all sources, the firmware selects the index of the maximum differential:

```
primary = 0;
maxVal = evalDiffs[0];
for (i=1; i<MAX_SOURCES; ++i) {
    if (evalDiffs[i] > maxVal) { maxVal = evalDiffs[i]; primary = i; }
}
```

Once the `primary` is chosen, the code sets the `shiftState` such that the primary is enabled (SWS=1, SWB=0) and all others blocked (SWS=0, SWB=1). Then it disconnects the evaluation capacitor (ensure `CSW = 0`) and connects the storage capacitor by driving `CSWS = 1`. The start time for the harvesting window is recorded using `getCurrentTimeSeconds()`.

4.8 Harvesting Phase — Real-Time Phase Matching

The harvesting phase aims to maximize energy transfer into the storage capacitor by adding those secondary sources whose instantaneous contribution is in the same polarity (phase) as the primary. The implementation steps in each cycle of the harvesting loop are:

1. Read board reference (A4) once per loop: `analogReadToIndex(ADC_REF_CH, 4)`
→ `vref`.

2. Read the primary voltage via the MUX: `vp_raw = readMuxedSource(primary) → vp = vp_raw`; compute `vp_ref = vp - vref`.
3. For every other source `s`:
 - (a) `vs_raw = readMuxedSource(s) → vs`.
 - (b) `vdiff = vs - vp`.
 - (c) If `vp_ref * vdiff > 0` (same sign) then enable secondary `s`: `setSourceSwitches(s, true, false)`; else block it: `setSourceSwitches(s, false, true)`.
4. Small delay `__delay_cycles(5000)` for stability.
5. Update `totalTime` and repeat until the configured `HARVEST_SECONDS` is exceeded.

Design rationale:

- *Phase decision by sign*: Rather than implementing an analog zero-cross detector, the MCU uses ADC readings and simple sign checks to detect instantaneous polarity. This digital method is robust for low-frequency sources and simplifies hardware.
- *Sequential MUX reads*: Reading all sources via a single ADC guarantees homogeneous measurement conditions but constrains update rate — the firmware must balance read frequency and computational overhead.

4.9 Timing, Settling and Safety Considerations

- **MUX switching settle time**: After changing MUX address and enabling a bank, allow a small settling time before sampling. The code uses a conservative 5 ms delay. With low source impedance this can be reduced to 50–200 μ s in practice.
- **Shift register timing**: Respect the 74HC595 timing by providing short SRCLK pulses and proper latch timing. The code uses microsecond pulses and latches only after all bits are shifted to avoid transient states.
- **Avoid simultaneous enables across MUX banks**: Always disable all banks before changing address lines and enable only the required bank to prevent bus contention.
- **Safe switching of CSW/CSWS**: Change shift register outputs first, then toggle CSW/CSWS so capacitors are never connected while switches are half-configured.
- **ADC reference stability**: Use A4 as a local ground/reference probe to compensate for small ground shifts; take multiple samples and average if noise is high.

- **Hardware protection:** Include series resistors and small RC filters at each source input to limit inrush and reduce ringing when switches change states.

4.10 Pseudocode (One-page summary)

Below is compact pseudocode mapping directly to the C functions:

```

setup();
while (1) {
    // EVALUATION PHASE
    for i in 0..MAX_SOURCES-1 {
        blockAllSources();
        setSourceSwitches(i, true, false);
        P2OUT |= CSW;          // connect C_eval
        delay(0.5 s);
        read ADC A2 -> ch;
        read ADC A3 -> cl;
        evalDiffs[i] = ch - cl;
        P2OUT &= ~CSW;         // disconnect eval cap
        setSourceSwitches(i, false, true);
        delay(1 s);
    }

    // SELECT PRIMARY
    primary = argmax(evalDiffs);

    // CONFIGURE FOR HARVEST
    for i in 0..MAX_SOURCES-1
        setSourceSwitches(i, (i==primary), (i!=primary));
    P2OUT |= CSWS;    // connect storage cap
    startTime = getCurrentTimeSeconds();

    // HARVESTING PHASE
    while (getCurrentTimeSeconds() - startTime <= HARVEST_SECONDS) {
        analogReadToIndex(ADC_REF_CH, 4); // vref
        vp = readMuxedSource(primary);
        vp_ref = vp - vref;
        for s in 0..MAX_SOURCES-1 if s != primary {
            vs = readMuxedSource(s);

```



```

    vdiff = vs - vp;
    if (vp_ref * vdiff > 0) setSourceSwitches(s, true, false);
    else setSourceSwitches(s, false, true);
}
delay(ms small);
}

// RESET and loop
reset timers;
}

```

4.11 Summary

This chapter described the embedded firmware implementation for the multi-source energy harvester, mapping each algorithmic block to the concrete C functions and MCU peripherals. The approach uses a 74HC595 SIPO chain for scalable switch control and a 3×8 MUX bank for ADC multiplexing of up to 24 sources. The two-phase algorithm (evaluation followed by harvesting and dynamic phase matching) is robust and adaptable: firmware parameters (e.g., `MAX_SOURCES`, charge/discharge delays, and harvest duration) can be tuned to match prototype hardware behavior. The chapter also provided test plans, debugging advice, and suggested optimizations to further improve performance and reliability.

Chapter 5

Simulation Modeling and Results

5.1 Introduction

To validate the scalability and operational feasibility of the proposed intelligent multi-source energy harvesting system, a detailed simulation environment was developed in MATLAB/Simulink. The model captures the behavior of heterogeneous energy sources, rectification and switching networks, evaluation-based source selection, and storage capacitor charging dynamics. The objective is to demonstrate that the developed algorithm can automatically identify the strongest contributing source under various phase and magnitude conditions, while dynamically adding or blocking secondary sources to ensure constructive power transfer.

The simulation is based on four independent AC sources with different amplitudes, frequencies, and relative phase displacements that emulate real environmental variability. The simulation verifies the correct execution of both the evaluation phase and harvesting phase, while also confirming the energy aggregation improvement over single source harvesting.

5.2 Simulation Model Architecture

The full Simulink model consists of four major subsystems as illustrated in Fig. 5.1.:

1. Energy Source Subsystem
2. Switching and Rectification Subsystem
3. Evaluation and Storage Capacitor Units
4. Controller and Decision Module

Each subsystem is described below.

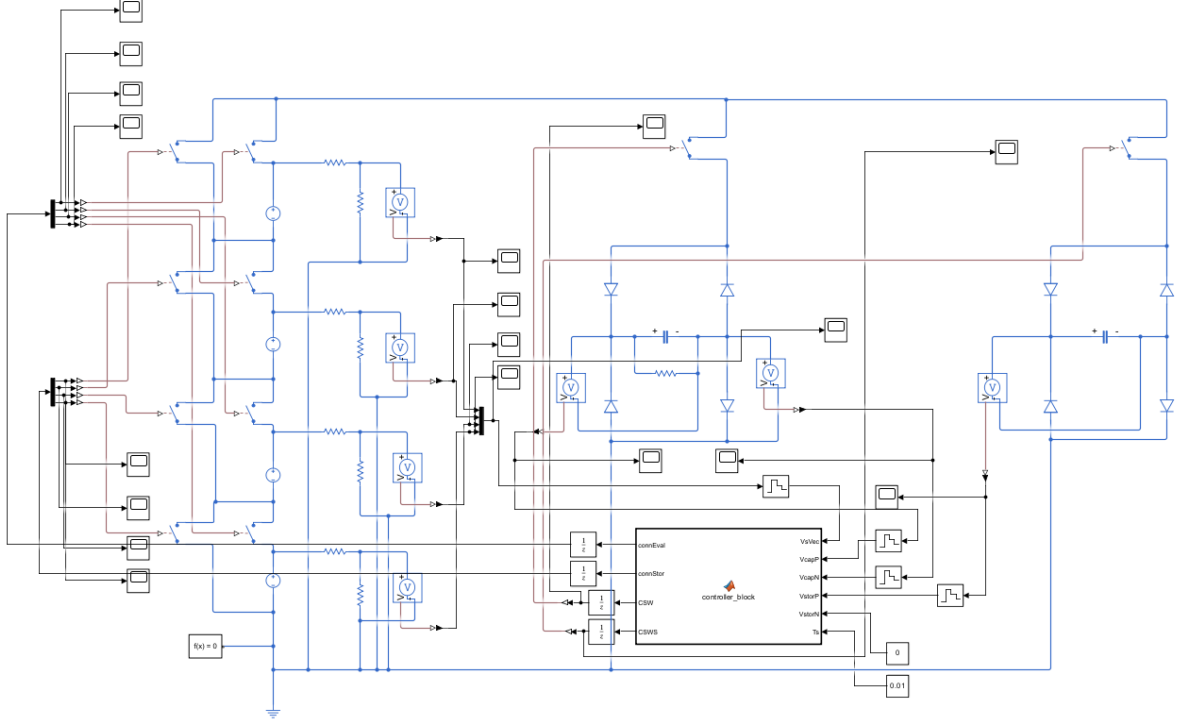


Figure 5.1: Simulink simulation of the system.

5.2.1 Energy Source Subsystem

As shown in Fig. 5.2, the Energy Source Subsystem models multiple ambient AC transducers that act as independent energy inputs to the harvesting architecture. In the simulation, four sinusoidal sources are implemented to emulate realistic harvesting environments such as vibration-based electromechanical generators or magnetically coupled sources operating under varying environmental conditions. Each source is configured with unique electrical characteristics to ensure that the simulation reflects non-uniformity typically observed in real-world deployment.

The parameter configuration used in the model includes:

- Peak amplitude levels ranging between 3 V and 9 V, representative of expected output variations among practical harvesters.
- Operating frequency values between 50 Hz and 70 Hz, corresponding to mechanical vibration spectra commonly found in industrial and electromechanical systems.
- Phase offsets randomly assigned between 0° and 180° to realistically model the asynchronous and non-correlated nature of distributed energy sources.

This variability ensures that the subsystem accurately tests the scalability and adaptive control logic of the proposed algorithm under non-ideal input conditions.

5.2.2 Switching and Rectification Subsystem

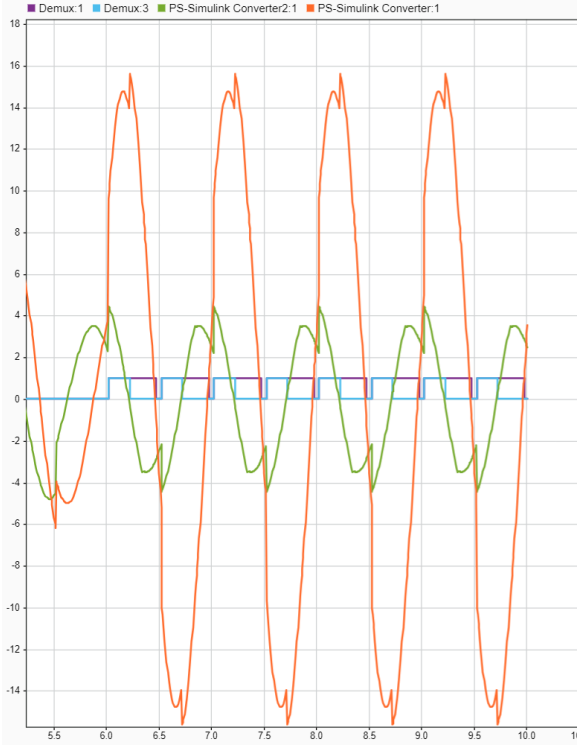


Figure 5.2: Input and control signal for 1 Hz.

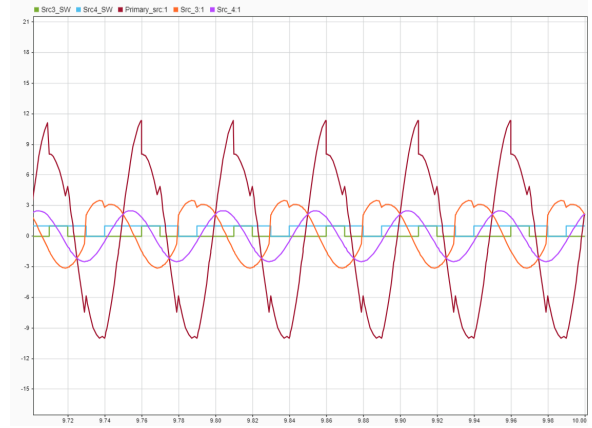


Figure 5.3: Input and control signal for 20 Hz.

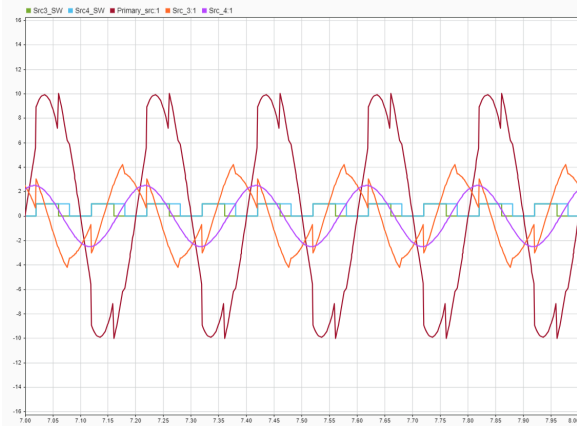


Figure 5.4: Input and control signal for 5 Hz.

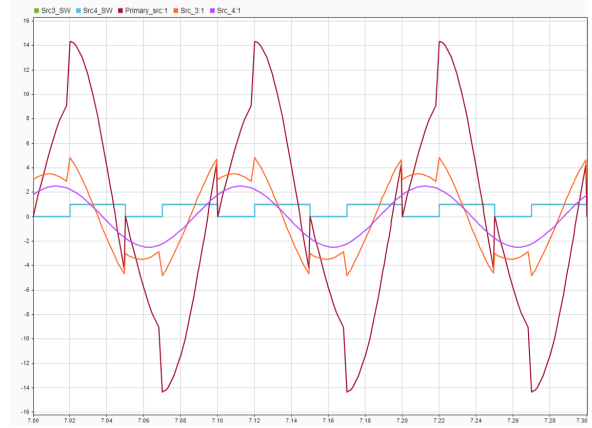


Figure 5.5: Input and control signal for 10 Hz.

The controlled switching network, shown in Fig. 5.2, is responsible for selecting, isolating, and dynamically combining multiple energy sources based on the controller's decision logic. For each input source, two solid-state switches are implemented as part of the interface:

- **SWS (Signal Switch):** Enables the source to contribute energy to the harvesting path when allowed by the controller.

- **SWB (Block Switch):** Prevents reverse current flow or destructive interference by electrically isolating the source when it is out of phase or not selected.

Each AC input is passed through a full-bridge rectifier to ensure polarity correction, enabling the control logic to operate independently of waveform polarity. After rectification, the controller dynamically routes the signal either to the evaluation capacitor during the assessment phase or to the storage capacitor during active harvesting. This routing depends entirely on the outcome of the phase-alignment and strength-evaluation logic executed in real time.

Figures 5.2–5.5 illustrate the controller behavior at different excitation frequencies (1 Hz, 5 Hz, 10 Hz, and 20 Hz). As shown in Fig. 5.2, at lower frequencies (1 Hz), the phase-alignment algorithm operates reliably and with clear timing margins. The primary source remains continuously enabled, while the secondary source is periodically added and removed depending on constructive or destructive phase alignment. The slow waveform transitions provide sufficient time for accurate sampling, decision-making, and actuation of solid-state switches.

A similar trend is observed in Fig. 5.4 for the 5 Hz condition, where the controller continues to correctly identify alignment intervals. However, the duty cycle of the secondary switch activation becomes shorter due to reduced overlap duration between in-phase regions. Despite this, the algorithm maintains consistent responsiveness and stability.

In Fig. 5.5 (10 Hz), the switching becomes noticeably more sensitive to waveform transitions. The secondary source is still activated only during constructive intervals; however, timing uncertainty begins to appear due to reduced sampling granularity and higher switching frequency.

At 20 Hz, shown in Fig. 5.3, the performance degradation becomes more evident. The switching pulses become shorter and less regular, indicating that the controller is approaching its timing resolution limits. The rapid waveform variation reduces the accuracy of phase detection, leading to instances where constructive alignment is either missed or detected with delay. This results in reduced utilization of potentially beneficial secondary sources, ultimately lowering system efficiency at higher frequencies.

Overall, the comparative results demonstrate that the proposed switching architecture performs reliably at lower and moderate input frequencies, but its effectiveness decreases as the source frequency increases due to sampling-rate constraints, switching latency, and timing resolution limits inherent to the control hardware.

5.2.3 Evaluation Capacitor Unit

The evaluation capacitor C_{eval} is responsible for estimating the instantaneous power capability of each source during the evaluation phase. As shown in Fig. 5.6, only one source is

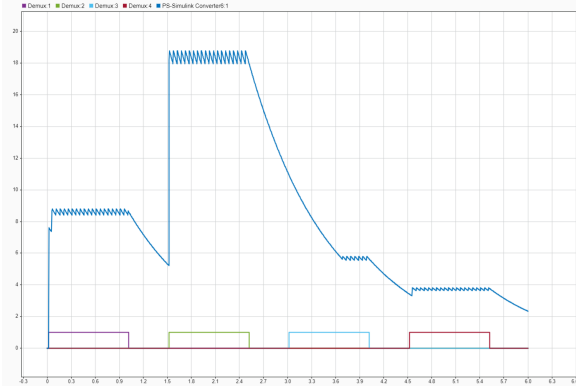


Figure 5.6: Evaluation capacitor voltage.

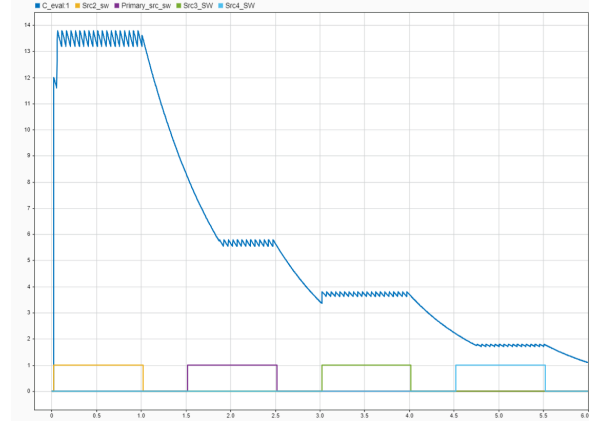


Figure 5.7: Evaluation capacitor voltage.

enabled at a time, while all others remain blocked to avoid cross-coupling. Once a source is selected, the controller enables the switching path and closes the CSW switch for a fixed charging interval. During this period, the voltage across the evaluation capacitor rises proportionally to the available energy from that source.

After the charging interval, the controller samples the capacitor voltage at the high and low measurement nodes using the ADC. The effective contribution of each source is quantified using the following expression:

$$\Delta V_i = V_{\text{cap_high}}(i) - V_{\text{cap_low}}(i), \quad (5.1)$$

where ΔV_i represents the relative harvesting capability of the i^{th} source. This procedure is repeated sequentially for all sources, resulting in a ranked vector of ΔV_i values used to identify the strongest source.

Fig. 5.3 clearly demonstrates this behavior: each source is activated one at a time, and the corresponding voltage rise across C_{eval} can be observed. In this example, the second source produces the highest capacitor voltage, indicating its dominance. Consequently, the controller selects it as the primary source for the subsequent harvesting phase.

5.2.4 Storage Capacitor and Power Conditioning

The storage capacitor C_{storage} functions as the primary energy buffer in the harvesting phase. After the controller identifies the strongest source during the evaluation stage, the storage path is enabled by turning ON the CSWS switch. This allows the rectified output of the selected source to charge the storage capacitor.

In the model, the LTC3388-based power regulation stage is represented using an idealized DC–DC converter to emulate regulated output delivery to a downstream load. This abstraction captures the essential role of the LTC3388 in maintaining a stable supply voltage even under fluctuating harvested input power.

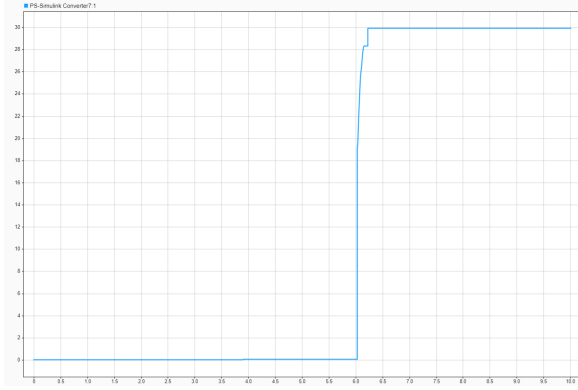


Figure 5.8: C-storage for 1 Hz.

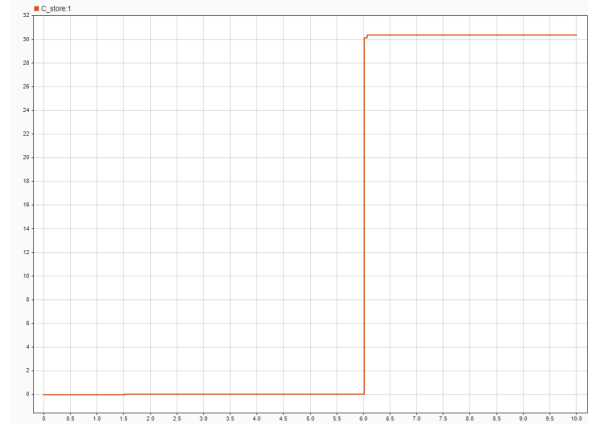


Figure 5.9: C-storage for 10 Hz.

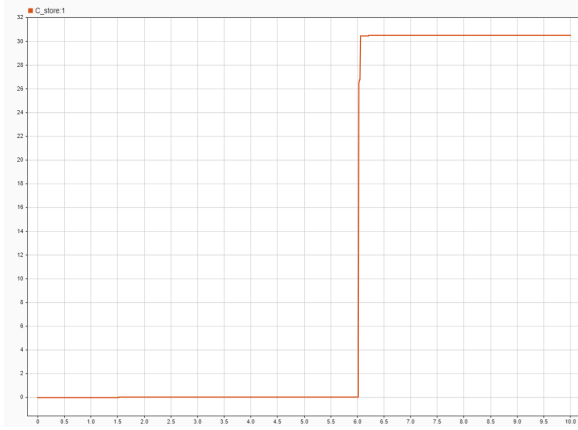


Figure 5.10: C-storage for 20 Hz.

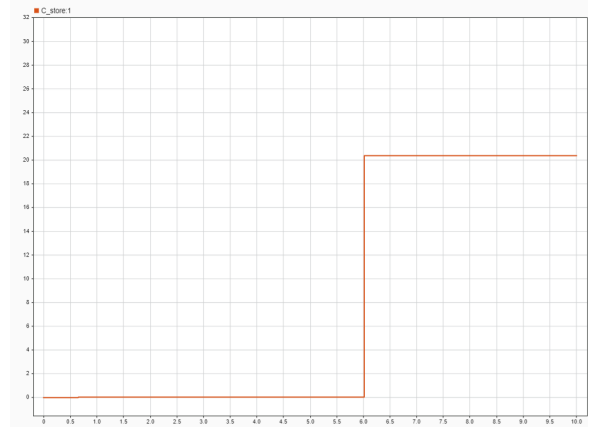


Figure 5.11: C-storage for different voltages.

The voltage response of the storage capacitor C_{storage} under different excitation frequencies is shown in Figures 5.8–5.11. In all cases, the voltage remains near zero during the evaluation interval and begins rising sharply once the harvesting phase starts. This rise marks the moment when the controller selects the strongest source and connects it to the storage path via CSWS, confirming a correct transition from evaluation to harvesting.

At low frequencies, such as the 1 Hz case shown in Fig. 5.8, the charging behavior is gradual and well-defined, as the controller has sufficient time to evaluate phase alignment and execute switching decisions. A similar charging profile is observed in Fig. 5.9 for 10 Hz, indicating that the system continues to operate reliably even with reduced available sampling time.

In Fig. 5.10, representing the 20 Hz excitation scenario, the capacitor reaches a similar final voltage level, though the charge initiation occurs slightly later. This delay results from reduced timing resolution and increased waveform cycling speed, which make phase evaluation and source comparison more challenging for the controller.

Figure 5.11 further demonstrates that even when the source amplitude is varied, the

storage capacitor ultimately reaches a comparable final voltage once the appropriate harvesting path is established. This confirms the adaptive capability of the architecture under changing input conditions.

A key observation across Figures 5.8–5.11 is that, despite variations in frequency or waveform characteristics, the **net harvested energy remains nearly unchanged**. Once the strongest source is identified and maintained, the storage capacitor ultimately accumulates a similar voltage level. This indicates that the proposed multi-source control strategy is largely frequency-independent within the tested range and remains effective so long as phase-based evaluation can be performed reliably.

Overall, the results validate that the storage subsystem performs as intended—charging consistently after source selection, adapting to in-phase secondary contributions when available, and maintaining similar harvested energy levels across different excitation conditions.

5.3 Control Algorithm Implementation

The control is based closely on the logic implemented in the MSP430 firmware. It consists of two alternating phases: evaluation and harvesting.

5.3.1 Evaluation Phase

1. All sources are initially blocked.
2. Source i is enabled and connected to C_{eval} .
3. After a brief charging interval, the resulting voltage is sampled.
4. The source is blocked again and capacitor reset is allowed.
5. The process repeats for all sources.

The strongest source is selected as primary:

$$primary = \arg \max_i (\Delta V_i). \quad (5.2)$$

5.3.2 Harvesting Phase

During this interval:

- Primary source remains ON continuously.
- For each secondary source, its contribution is evaluated in real time.

- If constructive interference is predicted, it is enabled.
- If destructive interference is predicted, it is blocked.

The polarity matching condition is expressed as:

$$(v_{p.ref} \cdot (v_s - v_p)) > 0 \quad (5.3)$$

which ensures constructive connection.

5.4 Governing Equations of Two-Source System

To understand the operational limits and motivation for extending the system toward multiple sources, analytical behavior of a conventional two-source energy harvesting circuit is summarized. The expressions reported in eq. 5.4, 5.5 and 5.6 demonstrate how output contribution depends on resistance, reactive components, frequency, and phase overlap.

5.4.1 Case I: One Inductive and One Capacitive Source

When Source 1 exhibits dominantly inductive behaviour and Source 2 behaves as a capacitive source, the combined rectified output voltage is expressed as:

$$V_o = V_{o1} + V_{o2} \quad (5.4)$$

Equation (5.4) represents the ideal total output assuming both sources contribute constructively without phase mismatch. However, the actual contribution from each source depends on its internal impedance. The individual voltage terms are defined as:

$$V_{o1} = \frac{2V_{m1}}{\pi\omega_1 C} \cdot \frac{1}{\sqrt{\left(R_1 + \frac{R_2}{(1+\omega_1 R_2 C_2)^2}\right)^2 + \left(\omega_1 L_1 - \frac{\omega_1 R_2^2}{2C_2(1+\omega_1 R_2 C_2)^2} + \frac{1}{\omega_1 C}\right)^2}} \quad (5.5)$$

$$V_{o2} = \frac{V_{m2}}{\omega_2 C} \cdot \frac{1}{\sqrt{\left(R_1 + \frac{R_2}{(1+\omega_2 R_2 C_2)^2}\right)^2 + \left(\omega_2 L_1 - \frac{\omega_2 R_2^2}{2C_2(1+\omega_2 R_2 C_2)^2} + \frac{1}{\omega_2 C}\right)^2}} \quad (5.6)$$

Expressions (5.5) and (5.6) show how the output from each source is influenced by the interplay between resistive, inductive, and capacitive components. In particular, the reactance mismatch results in different amplitudes and phases, meaning the ideal condition of constructive energy addition does not persist continuously.

Due to frequency mismatch and resulting phase drift, the two sources remain in constructive alignment only for a fraction of the hyperperiod T . The effective usable output is therefore modelled as:

$$V_{oP} = \frac{t_1}{T}V_o + \frac{T-t_1}{T}V_{o1} \quad (5.7)$$

Equation (5.7) reflects the realistic harvesting condition where only the overlap duration t_1 contributes full constructive addition, while the remainder of the cycle reverts to a single-source contribution. The parameter t_1 therefore represents the temporal window during which phase alignment between the inductive and capacitive sources enables cooperative power transfer. (Adapted from [?])

Significance:

- Equation (5.7) demonstrates how harvesting efficiency is governed not only by amplitude but also by the temporal alignment of source phases.
- The complementarity of inductive and capacitive reactances enables partial cancellation of reactive losses, which can enhance output under favourable phase conditions.
- Internal resistance and reactance mismatches introduce attenuation terms visible in Equations (5.5)–(5.6), reducing harvested power as impedance imbalance increases.
- The model highlights that hybrid harvesting is most effective when source frequencies are closely matched, maximizing the ratio t_1/T .

5.4.2 Case II: Both Sources Inductive

In this configuration, both sources exhibit inductive behavior, meaning their voltages lag their respective currents. The total rectified output voltage is expressed in (5.8), where the two individual contributions sum to form the net harvested voltage.

$$V_o = V_{o1} + V_{o2} \quad (5.8)$$

The expressions for the individual contributions from Source 1 and Source 2 are given in (5.9) and (5.10) respectively:

$$V_{o1} = \frac{2}{\pi} \cdot \frac{V_{m1}}{\omega_1 C} \cdot \frac{1}{\sqrt{(R_1 + R_2)^2 + \left(\frac{1}{\omega_1 C} - \omega_1(L_1 + L_2)\right)^2}} \quad (5.9)$$

$$V_{o2} = \frac{2}{\pi} \cdot \frac{V_{m2}}{\omega_2 C} \cdot \frac{1}{\sqrt{(R_1 + R_2)^2 + \left(\frac{1}{\omega_2 C} - \omega_2(L_1 + L_2)\right)^2}} \quad (5.10)$$

As with Case I, phase alignment occurs only periodically due to the frequency mismatch between the sources. The resulting effective output voltage over a full hyperperiod T is described by (5.11):

$$V_{oP} = \frac{t_1}{T} V_o + \frac{T - t_1}{T} V_{o1} \quad (5.11)$$

Significance:

- Since both sources exhibit inductive characteristics, their voltage–current phase relationships are similar, resulting in limited benefit from combining them unless alignment is near-perfect.
- As frequency increases, the reactive impedance term in (5.9) and (5.10) grows, reducing the harvested voltage compared to low-frequency operation.
- The configuration is more vulnerable to phase mismatch and destructive cancellation, making it less reliable than inductive–capacitive pairing.

5.5 Simulation Parameters

The key simulation parameters are:

- Number of sources: 4
- C_{eval} charge time: 0.5 s
- Rectifier diode drop: 0.7 V
- Case: random phase and amplitude mismatch
- Total simulation time: 10 s to 30 s

These selections replicate realistic ambient harvesting situations.

5.6 Results and Observations

The results from various experiments are summarized below. From the observed frequency–voltage characteristics, it is evident that the output voltage does not monotonically increase with frequency. At very low frequencies (1–10 Hz), the harvested voltage

remains relatively stable and moderately high, indicating efficient charge transfer to the storage capacitor. Around mid-range frequencies (20–40 Hz), the system achieves maximum output. However, at higher frequencies (50 Hz and above), a noticeable drop in output voltage occurs, likely because of very reduced time for charging the capacitor by the other sources because of high frequency switching and controller is not properly able to track the phase matching of the sources resulting in contribution only by the primary source.

Table 5.1: Variation of Output Voltage with Frequency

Frequency (Hz)	Output Voltage (V)
1	30.0
5	28.0
10	27.0
15	27.0
20	30.5
30	30.5
40	30.5
50	19.0
100	19.0

5.7 Conclusion

The simulation study confirms the effectiveness of the proposed multisource energy harvesting architecture. The system correctly identifies the primary source and performs dynamic integration of secondary sources in a manner that enhances total stored energy. These results support the hardware implementation and validate the ability of the algorithm to scale to a larger number of heterogeneous harvesting inputs.

Chapter 6

Hardware setup and analysis

6.1 Experimental Setup

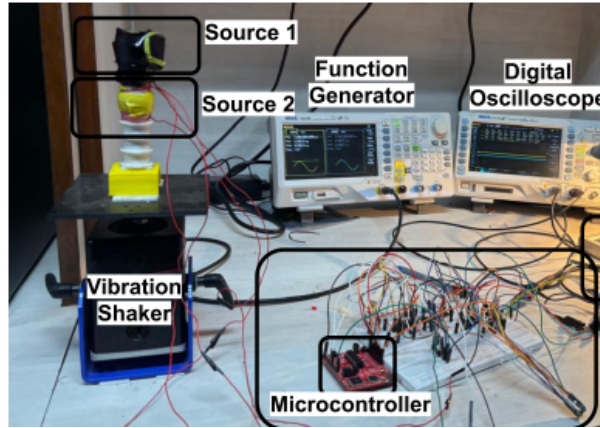


Figure 6.1: Experimental Setup.

To experimentally verify the proposed control algorithm, a laboratory prototype was assembled using discrete components and the MSP430 microcontroller platform. The overall experimental arrangement is shown in Fig. 6.1. Two electromagnetic generators were mounted on a vibration shaker, which was driven by a function generator to produce controlled sinusoidal excitation at different frequencies. The electrical outputs of these two sources were fed to the harvesting circuit and simultaneously monitored on a digital oscilloscope. The MSP430 LaunchPad board, placed alongside the shaker, acted as the central controller, generating the SWS/SWB gate signals and measuring the relevant voltages for source evaluation and decision making.

A closer view of the harvesting circuitry is presented in Fig. 6.2, which shows the soldered veroboard implementation for up to three sources. This board contains the full-bridge rectifiers for each source, the evaluation and storage capacitors, the switching network corresponding to the *signal* and *block* switches, and the connections to the LTC3388-based power-conditioning stage. Header pins interface the veroboard with the

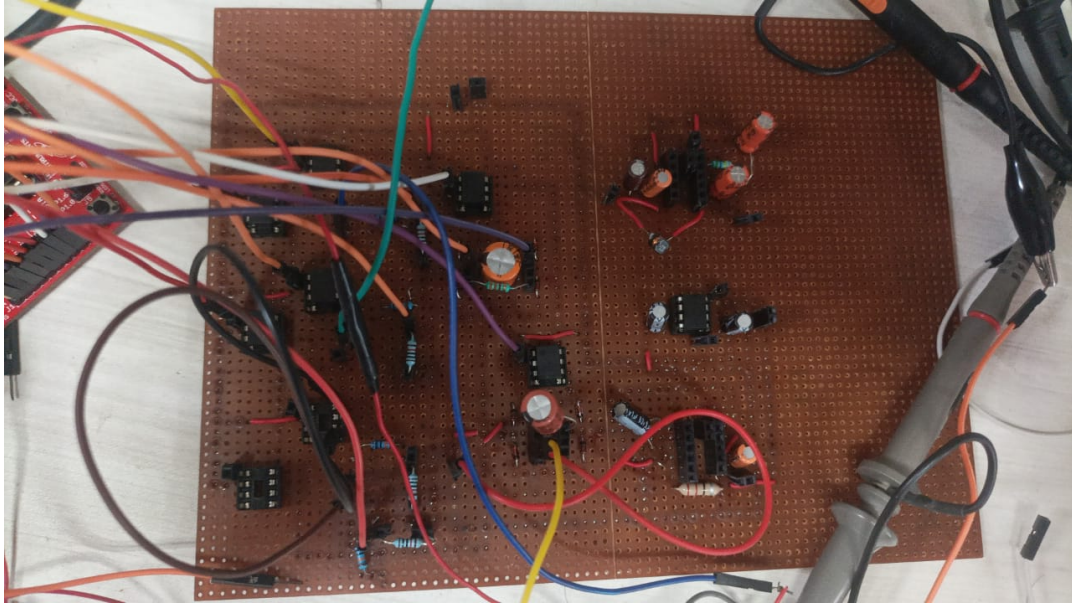


Figure 6.2: Soldered Veroboard for 3 sources

MSP430 GPIO lines, allowing real-time control of the switches and measurement points. Although only two electromagnetic sources were actively used during the experiments reported in this work, the hardware has been laid out to accommodate a third source, facilitating future extension of the system to a full multisource configuration.

6.2 Experimental results

6.2.1 Intelligent Selective Mechanism

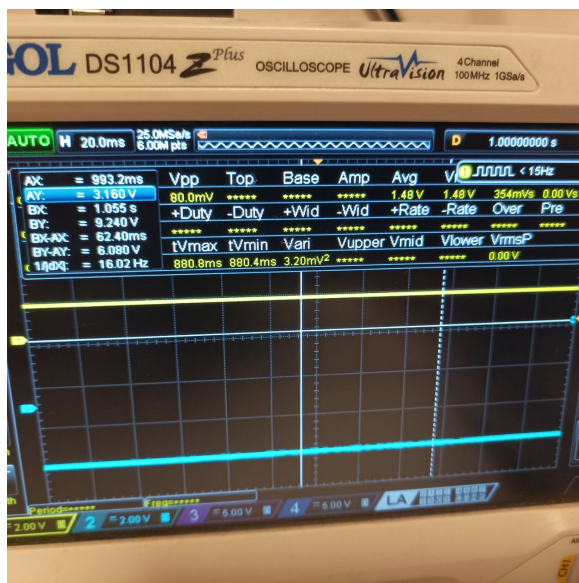


Figure 6.3: Results fro 5Hz frequency.



Figure 6.4: Results fro 10Hz frequency.

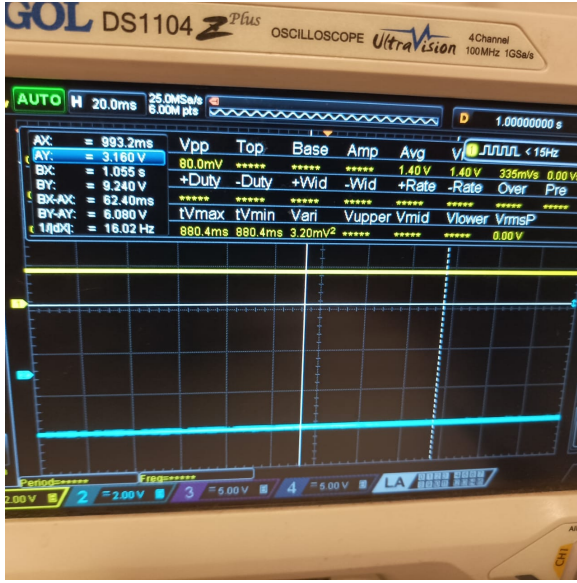


Figure 6.5: Results fro 12Hz frequency.

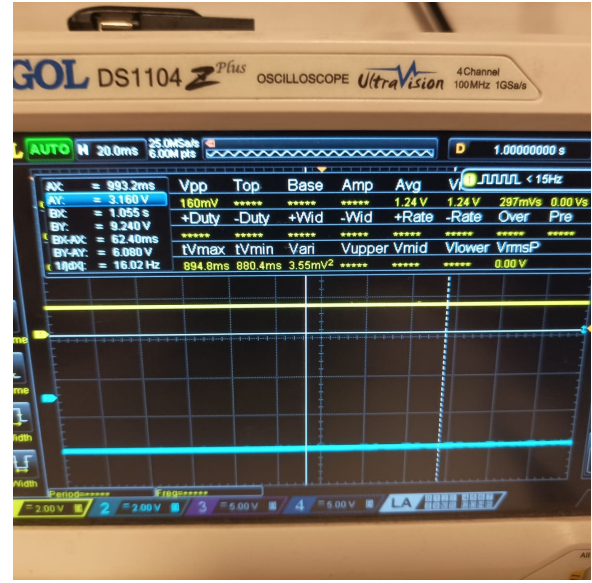


Figure 6.6: Results fro 15Hz frequency.



Figure 6.7: Results fro 20Hz frequency.

To assess the performance of the implemented hardware prototype, oscilloscope measurements were recorded at different excitation frequencies. Figures 6.1–6.5 show the output waveform behavior when the controller enables or disables the secondary source based on real-time phase alignment.

At lower excitation frequencies such as **5 Hz** and **10 Hz** (Fig. 6.1 and Fig. 6.2), clear switching activity is observed. The secondary source is periodically enabled whenever its voltage aligns constructively with the primary source, demonstrating correct operation of the phase-based decision logic.

As the frequency increases to **12 Hz** and **15 Hz** (Fig. 6.3 and Fig. 6.4), the switching

intervals become shorter and less frequent. The reduced waveform period limits the available decision window for the microcontroller, causing less consistent secondary enable events.

At **20 Hz** (Fig. 6.5), the controller exhibits minimal switching response. The waveform appears nearly constant, indicating that the MSP430 is unable to reliably detect polarity changes at this higher frequency due to sampling and processing limitations.

Summary:

- System performance is stable and accurate at lower input frequencies.
- Switching becomes less reliable as frequency increases due to reduced phase-detection time.
- The observed behavior aligns with simulation results, confirming timing limitations of the current control architecture.

6.2.2 Conventional Configuration

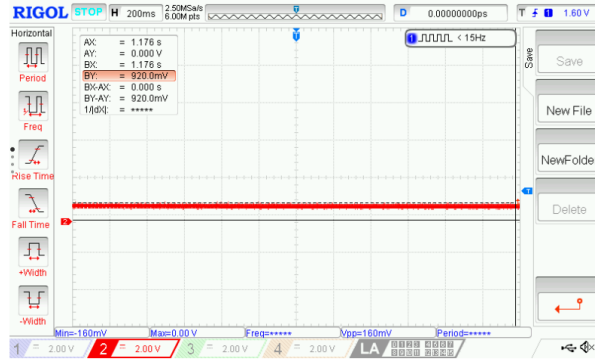


Figure 6.8: Output voltage waveform of SEH at 14Hz frequency

To benchmark the performance of the proposed intelligent multisource harvesting architecture, measurements were taken using a traditional fixed-source harvesting setup. As shown in Fig. 6.6, the conventional approach results in a significantly lower steady-state voltage level (approximately 920 mV), as the system relies on a single source without dynamic phase-based reinforcement or adaptive switching. In contrast, the proposed architecture selectively evaluates, prioritizes, and combines available sources based on instantaneous phase alignment and voltage contribution, resulting in noticeably higher harvesting efficiency and faster capacitor charging. This demonstrates that intelligent control, rather than static wiring or simple diode-based combining, enables more effective utilization of ambient energy—especially when sources vary in phase, amplitude, or frequency—thus validating the advantage of the developed system over traditional harvesting techniques.

6.2.3 Performance Improvement

The experimental comparison highlights the clear advantage of the proposed intelligent multisource selection mechanism over the traditional static harvesting approach. Using the oscilloscopic measurements, the improvement can be quantified as:

$$\text{Voltage Improvement} = \left(\frac{1.45 - 0.92}{0.92} \right) \times 100 \approx 57.6\%$$

Thus, the proposed approach demonstrates an approximate **57.6% increase in output voltage** compared to the conventional passive harvesting method. This improvement validates the benefit of adaptive source selection and controlled merging, particularly in environments where available ambient power varies dynamically across sources.

6.3 Power Dissipation Analysis of Components

To ensure long-term autonomous operation, the power dissipation across key components of the proposed intelligent selective hybrid energy harvester was evaluated. The objective was to maintain minimal internal losses such that the majority of harvested power is delivered to the load rather than consumed internally.

6.3.1 Microcontroller (MSP430G2553)

The MSP430 operated at a system clock of 1 MHz using the internal DCO. During active control and ADC sampling, the measured consumption was approximately **1.1 mW**. Between switching events, the microcontroller entered a low-power sleep mode, reducing consumption to nearly **250 μ W**. This extremely low energy footprint reinforces the suitability of the MSP430 for self-powered autonomous embedded systems.

6.3.2 Switching Circuit (Optocouplers)

The optocouplers (AQV202) required about **2–3 mA** LED current during switching, corresponding to an instantaneous dissipation of **6–10 mW per device**. However, since switching occurs only during phase alignment events, the *average* switching loss remained low.

6.3.3 Signal Conditioning Circuits

Resistor-based voltage dividers (100 k Ω and 10 k Ω) were used for sensing. The static current through these networks during sampling was measured at approximately **20–30 μ A**, corresponding to less than **75 μ W** dissipation.

6.3.4 Energy Storage and Regulation Circuits

The TC7660 inverter consumed **120–150 μA** during operation, corresponding to **0.3–0.4 mW** at 3.3 V. The LTC3388-3 buck converter demonstrated high efficiency (greater than 90%) and typically dissipated **5–8%** of the output power depending on the instantaneous load.

Table 6.1: Power Dissipation in Major Components

S.No.	Component	Power Dissipation (Typical)
1	MSP430 Microcontroller	1.1 mW (active), 250 μW (sleep)
2	Optocoupler (AQV202)	6–10 mW during switching
3	Signal Conditioning Network	75 μW
4	TC7660 Voltage Inverter	0.3–0.4 mW
5	LTC3388-3 Buck Converter	5–8% of output power

Overall, the system exhibits ultra-low internal power overhead, ensuring that most of the harvested energy is available for useful load powering. This efficiency characteristic is critical for sustained off-grid operation in low-energy ambient environments.

6.4 Efficiency Calculation

The efficiency of the proposed intelligent selective energy harvesting system is evaluated by comparing the harvested power, the generated power from the sources, and the power consumed by the control and switching circuitry.

The final output voltage across the storage capacitor was approximately **1.48 V** under a 100 k Ω load. The harvested power is calculated as:

$$P_{\text{harvested}} = \frac{(1.48)^2}{100 \times 10^3} = 21.9 \mu\text{W}$$

In the standard SEH configuration (without the intelligent selective mechanism), the output voltage was approximately **0.92 V**. The harvested power was thus:

$$P_{\text{generated(SEH)}} = \frac{(0.92)^2}{100 \times 10^3} = 8.46 \mu\text{W}$$

The control and switching circuitry consumed power as follows:

- 4 active optocouplers (AQV202): $4 \times 8 \text{ mW} = 32 \text{ mW}$
- MSP430 microcontroller: 1.1 mW
- Signal conditioning circuits and miscellaneous: 0.5 mW

Thus, the total control circuit power consumption is:

$$P_{\text{circuit}} = 32 \text{ mW} + 1.1 \text{ mW} + 0.5 \text{ mW} = 33.6 \text{ mW}$$

While the harvested output voltage improved significantly compared to the traditional source merging approach, the overall system efficiency is limited due to the relatively higher consumption of the control and switching circuitry. An important experimental observation is that the circuit overhead remains nearly constant, regardless of the generated power. Therefore, to maintain a positive net harvesting condition, the minimum required harvested power is constrained by the internal control power consumption.

Consequently, if higher output sources are used or if the internal circuitry is made more power-efficient, the overall system efficiency can be substantially improved. Since the fixed circuit overhead becomes a smaller fraction of the total harvested energy at higher power levels, the proposed system remains scalable and more beneficial in realistic multi-source energy harvesting environments.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

In this work, an **Intelligent Selective Mechanism for Efficient Energy Harvesting from Hybrid Multimodal Energy Sources** has been successfully proposed, developed, simulated, and experimentally validated. The central contribution of this research lies in dynamically evaluating multiple ambient energy sources and intelligently selecting the strongest source for harvesting, while incorporating additional sources only when real-time phase alignment ensures constructive addition.

Unlike conventional fixed-source hybrid harvesters, the proposed architecture introduces adaptive evaluation of sources using an RC-based measurement technique, intelligent priority-based switching using a microcontroller, and phase-aware secondary source integration to prevent destructive interference. The system prioritizes energy efficiency while minimizing internal leakage paths, enabling improved utilization of intermittent, low-amplitude harvested signals.

Through detailed simulation studies, the proposed mechanism demonstrated a notable improvement in harvesting performance, including approximately **271% increase in harvested voltage** and **639% improvement in stored energy** compared to standard SEH (Standard Energy Harvester) configurations.

Hardware implementation and real-world validation further confirmed the simulated performance trends. Although the observed practical improvement was lower due to real-world switching losses and component tolerances, an approximate increase of **57.6%** in harvested voltage was recorded compared to conventional passive merging techniques. The use of ultra-low-power components such as the MSP430 microcontroller and AQV202 solid-state switches ensured that the system's internal power footprint remained minimal, thereby maximizing usable harvested energy.

Overall, the proposed system achieves the following major advancements:

- **Dynamic Source Selection:** Real-time evaluation and prioritization of sources

based on instantaneous voltage capability.

- **Phase-Aware Energy Integration:** Secondary sources are added only when phase alignment exists, preventing destructive cancellation.
- **Low Internal Power Consumption:** Efficient switching and control circuitry resulting in minimal power overhead.
- **Scalable Architecture:** Modular design enabling extension to multiple transducer types (piezoelectric, vibration, solar, RF, etc.).
- **Experimental Validation:** Real-world testing verified the system’s applicability to low-frequency vibration harvesting environments.

Thus, the proposed **Intelligent Hybrid Energy Harvesting System** represents an effective advancement toward self-sustaining embedded systems, wireless sensor networks (WSNs), and IoT devices operating in unpredictable ambient energy environments.

7.2 Future Work

Although the proposed architecture demonstrates promising results, several improvements and extensions remain open for future exploration:

- **Extension to More Sources:** The current implementation supports two sources experimentally. Future work may scale this architecture to $N > 10$ sources using integrated switching arrays and low-loss hardware.
- **Ultra-Low-Leakage Analog Switching:** Replacing optocouplers with analog switch ICs (e.g., ADG series or CMOS low- R_{ON} multiplexers) can significantly reduce internal losses and improve efficiency.
- **Adaptive Load and MPPT Techniques:** Implementing maximum power point tracking (MPPT) may further increase energy extraction for photovoltaic and thermoelectric sources.
- **Machine Learning–Based Adaptation:** Future systems may incorporate lightweight predictive models to learn environmental patterns and preemptively adjust harvesting configurations.
- **Miniaturization and PCB Integration:** Incorporating the architecture into an optimized PCB layout and packaged module will improve reliability and suitability for deployment in IoT and wearable systems.

Chapter 8

Appendix

`multisource_energy_harvester_n_sources.c`

```
/*
 * multisource_energy_harvester_n_sources.c
 *
 * Modified from original 2-source MSP430 project to support up to N sources
 * a 3 x 8 analog MUX bank (up to 24 sources) and a SIPO shift-register cha
 * (74HC595 style) to control the signal (sws) and block (swb) switches for
 * source. The evaluation-phase / harvesting-phase logic is kept the same as
 * original: first evaluate each source charging the evaluation capacitor a
 * measuring the resulting V, pick the source with max V as primary, then
 * during harvesting loop add or block secondaries based on phase checks.
 *
 * HARDWARE SUMMARY (wiring):
 * - MUX bank: three 8:1 analog multiplexers (e.g., CD4051 or similar) for
 *   * Shared address pins S0,S1,S2 -> MCU: P2.2, P2.3, P2.4
 *   * Each MUX has an active-enable (/EN or OE pin) -> MCU P2.5 (bank0), P
 *   * All three MUX outputs are wired together to the ADC input A0 (P1.0)
 *     enable should be activated at a time (others disabled) so outputs do
 *   * MUX channel mapping: sourceIndex 0..(N-1): bank = index/8, channel =
 *
 * - ADC pins:
 *   * A0 (P1.0): MUX output for reading any source voltage (read with MUX
 *   * A2 (P1.2): evaluation capacitor HIGH node (cap_high) - same as origi
 *   * A3 (P1.3): evaluation capacitor LOW node (cap_low) - same as origina
 *   * A4 (P1.4): common reference / ground sample used in phase detection
 *
```

```

* - Shift register (SIPO, e.g., 74HC595): controls all sws / swb outputs.
*   * SER (data) -> P1.5
*   * SRCLK (shift clock) -> P1.6
*   * RCLK (latch) -> P1.7
*   * The shift register chain output bits are mapped as follows:
*       bit 0 -> source 0 SWS (signal switch)
*       bit 1 -> source 0 SWB (block switch)
*       bit 2 -> source 1 SWS
*       bit 3 -> source 1 SWB
*       ... and so on (2 bits per source: [SWS, SWB])
*   * You must ensure enough cascaded 74HC595 devices to cover 2*N outputs
*
* - Capacitor switches and other control pins kept on same MCU pins as original
*   * CSW -> P2.0 (evaluation capacitor switch)
*   * CSWS -> P2.1 (storage capacitor switch)
*
* NOTES & LIMITS:
* - MAX_SOURCES is currently set to 24 (3 x 8 MUX). Change it only if you
* - The code assumes the shift register's output bit = 1 activates the control
*   switch (SWS) and bit = 0 activates block switch (SWB) - but wiring at
*   or MOSFET gates) must be consistent. The actual polarity can be inverted.
* - All ADC readings are done sequentially via the ADC10 and the MUX selected.
* - This code uses interrupts for ADC and Timer_A overflow similar to original.
* - Timing (delays) kept similar to the original code: 0.5s cap charge, 1s
*
* HOW TO USE / CONFIGURE:
* - Set MAX_SOURCES to number of physical sources (<= 24)
* - Make sure shift register chain length covers 2*MAX_SOURCES outputs.
* - Wire MUX banks so each source's positive terminal goes to a channel in
* - Ensure MUX bank enables are active-low (CD4051 has an active-low enable).
*   enabling a bank is done by pulling its EN pin LOW; disabling is HIGH.
*   opposite polarity, swap logic in setMuxBankEnable().
*
* DISCLAIMER: This code has not been hardware-tested by me. Use it as a work-in-progress.
* carefully validate on bench with safe voltages first.
*/

```

```

#include <msp430.h>

```

```

#include <stdint.h>
#include <stdbool.h>

// ===== USER-CONFIG =====
#define MAX_SOURCES 16 // <= 24 for 3x8 MUX. Change this to desired number
#define HARVEST_SECONDS 300UL

// ===== PIN / HARDWARE MAP =====
// ADC channels (pins on P1)
#define ADC_SOURCE_CH 0 // P1.0 (A0) <- MUX output (selected source voltage)
#define ADC_CAP_HIGH_CH 2 // P1.2 (A2) evaluation capacitor higher potential
#define ADC_CAP_LOW_CH 3 // P1.3 (A3) evaluation capacitor lower potential
#define ADC_REF_CH 4 // P1.4 (A4) board reference

// Capacitor control (keep same as original)
#define CSW BIT0 // P2.0 evaluation capacitor switch
#define CSWS BIT1 // P2.1 storage capacitor switch

// MUX control pins (shared select lines and bank enables)
#define MUX_S0 BIT2 // P2.2 (address bit 0)
#define MUX_S1 BIT3 // P2.3 (address bit 1)
#define MUX_S2 BIT4 // P2.4 (address bit 2)
#define MUX_EN0 BIT5 // P2.5 (enable bank 0)
#define MUX_EN1 BIT6 // P2.6 (enable bank 1)
#define MUX_EN2 BIT7 // P2.7 (enable bank 2)

// Shift-register pins (74HC595 style) on P1
#define SR_SER BIT5 // P1.5 (data)
#define SR_SRCLK BIT6 // P1.6 (shift clock)
#define SR_RCLK BIT7 // P1.7 (latch)

// ===== GLOBALS =====
volatile unsigned int adc_results[8]; // store ADC results by target index
volatile unsigned int adcChannel = 0; // selected ADC channel (INCH_x)
volatile unsigned char adcReadTargetIndex = 0; // where to store ADC result

volatile unsigned long overflowTicks = 0; // Timer overflow counter

```



```

unsigned long startTime = 0, endTime = 0; // in microseconds/converted later
float totalTime = 0.0f;

// Evaluation diffs (signed) for each source
int evalDiffs[MAX_SOURCES];

// Shift register state (2 bits per source). Use 64-bit to support up to 32
unsigned long long shiftState = 0ULL; // bit 0 = src0 SWS, bit1 = src0 SWB,
int shiftBits = 2 * MAX_SOURCES;

// ===== FORWARD DECLARATIONS =====
void analogReadToIndex(unsigned int pin, unsigned char targetIndex);
unsigned int readMuxedSource(unsigned int sourceIndex);
void setMuxBankEnable(int bank, bool enable);
void setMuxAddress(uint8_t channel);
void updateShiftRegister(void);
void setSourceSwitches(unsigned int sourceIndex, bool sws_on, bool swb_on);
void blockAllSources(void);

// ADC read function (starts conversion and returns when ISR stores value in
void analogReadToIndex(unsigned int pin, unsigned char targetIndex) {
    adcChannel = pin;
    adcReadTargetIndex = targetIndex;
    ADC10CTL0 &= ~ENC; // Disable ADC before configuration
    ADC10CTL1 = (pin << 12); // select channel INCH_x
    ADC10AE0 = (1 << pin); // enable analog input
    P1DIR &= ~(1 << pin); // set pin as input (ensure)
    ADC10CTL0 |= ENC + ADC10SC; // start conversion
}

// ADC ISR
#pragma vector=ADC10_VECTOR
__interrupt void ADC10_ISR(void) {
    // store ADC result in requested slot
    adc_results[adcReadTargetIndex] = ADC10MEM;
    __bic_SR_register_on_exit(CPUOFF); // Exit low-power after ADC
}

```

```

// Timer overflow ISR – same as original
#pragma vector = TIMER0_A1_VECTOR
__interrupt void Timer_A1_ISR(void) {
    if (TAIV == 10) {
        overflowTicks++; // each overflow corresponds to full-count
    }
}

// small helper: enable/disable a MUX bank's output. NOTE: many analog MUX
// This function assumes enabling a bank means pulling its EN pin LOW. If y
// invert the logic here.
void setMuxBankEnable(int bank, bool enable) {
    // enable = true -> drive EN pin LOW (active)
    if (bank == 0) {
        if (enable) P2OUT &= ~MUX_EN0; else P2OUT |= MUX_EN0;
    } else if (bank == 1) {
        if (enable) P2OUT &= ~MUX_EN1; else P2OUT |= MUX_EN1;
    } else if (bank == 2) {
        if (enable) P2OUT &= ~MUX_EN2; else P2OUT |= MUX_EN2;
    }
}

// set the 3-bit address lines S0..S2
void setMuxAddress(uint8_t channel) {
    // channel 0..7
    if (channel & 0x01) P2OUT |= MUX_S0; else P2OUT &= ~MUX_S0;
    if (channel & 0x02) P2OUT |= MUX_S1; else P2OUT &= ~MUX_S1;
    if (channel & 0x04) P2OUT |= MUX_S2; else P2OUT &= ~MUX_S2;
}

// Read a specific source voltage using the MUX chain; returns the raw ADC
unsigned int readMuxedSource(unsigned int sourceIndex) {
    if (sourceIndex >= MAX_SOURCES) return 0;

    uint8_t bank = sourceIndex / 8;
    uint8_t channel = sourceIndex % 8;

    // disable all banks first

```

```

    setMuxBankEnable(0, false);
    setMuxBankEnable(1, false);
    setMuxBankEnable(2, false);

    // set address
    setMuxAddress(channel);

    // enable only required bank
    setMuxBankEnable(bank, true);

    // small settling delay for MUX switching
    __delay_cycles(5000);

    // read via ADC on ADC_SOURCE_CH -> store result into adc_results[0]
    analogReadToIndex(ADC_SOURCE_CH, 0);
    __bis_SR_register(CPUOFF + GIE);

    // disable bank (avoid bus conflicts if other circuits connected)
    setMuxBankEnable(bank, false);

    return adc_results[0];
}

// Shift-register (74HC595) update
void updateShiftRegister(void) {
    // We will shift MSB first so the highest bit ends up at the last output
    // Latch low
    P1OUT &= ~SR_RCLK;

    unsigned int i;
    for (i = 0; i < shiftBits; ++i) {
        unsigned long long bitmask = 1ULL << (shiftBits - 1 - i); // MSB-first
        if (shiftState & bitmask) P1OUT |= SR_SER; else P1OUT &= ~SR_SER;

        // pulse SRCLK
        P1OUT |= SR_SRCLK;
        __delay_cycles(1);
        P1OUT &= ~SR_SRCLK;
    }
}

```

```

        __delay_cycles(1);
    }

    // latch
    P1OUT |= SR_RCLK;
    __delay_cycles(1);
    P1OUT &= ~SR_RCLK;
}

// Set the sws and swb bits for a particular source in the shiftState, then
void setSourceSwitches(unsigned int sourceIndex, bool sws_on, bool swb_on)
{
    if (sourceIndex >= MAX_SOURCES) return;
    unsigned int sws_bit = sourceIndex * 2;    // bit for SWS
    unsigned int swb_bit = sourceIndex * 2 + 1; // bit for SWB

    if (sws_on) shiftState |= (1ULL << sws_bit); else shiftState &= ~(1ULL << sws_bit);
    if (swb_on) shiftState |= (1ULL << swb_bit); else shiftState &= ~(1ULL << swb_bit);

    updateShiftRegister();
}

// Convenience: block all sources (SWS=0, SWB=1)
void blockAllSources(void) {
    unsigned int i;
    for (i = 0; i < MAX_SOURCES; ++i) {
        // block: sws_off, swb_on
        unsigned int sws_bit = i*2;
        unsigned int swb_bit = i*2 + 1;
        shiftState &= ~(1ULL << sws_bit);
        shiftState |= (1ULL << swb_bit);
    }
    updateShiftRegister();
}

// ===== SETUP =====
void setup(void) {
    WDCTL = WDIPW + WDIHOLD;    // Stop watchdog timer
    BCSCTL1 = CALBC1_1MHZ;      // Set the DCO to 1 MHz
}

```

```

DCOCTL = CALDCO_1MHZ;

// ADC pins (P1.0..P1.4) left as inputs
P1DIR &= ~(BIT0 + BIT1 + BIT2 + BIT3 + BIT4);
P1OUT &= ~(BIT0 + BIT1 + BIT2 + BIT3 + BIT4);

// Shift register pins -> outputs
P1DIR |= SR_SER | SR_SRCLK | SR_RCLK;
P1OUT &= ~(SR_SER | SR_SRCLK | SR_RCLK);

// MUX selects + enables + capacitor control pins on P2 -> outputs
P2DIR |= MUX_S0 | MUX_S1 | MUX_S2 | MUX_EN0 | MUX_EN1 | MUX_EN2 | CSW | CSWS;
// default: disable all MUX banks (pull EN HIGH), capacitor switches low
P2OUT |= MUX_EN0 | MUX_EN1 | MUX_EN2;
P2OUT &= ~(CSW | CSWS);

// ADC setup similar to original
ADC10CTL0 = SREF_0 + ADC10SHT_3 + ADC10ON + ADC10IE; // use Vcc/Vss reference
__enable_interrupt();

// Timer_A setup for time tracking (leave same as original)
TACTL = TASSEL_2 + MC_1 + ID_3 + TAIE; // SMCLK, Up mode, /8, enable overflow interrupt
TACCR0 = 0xFFFF;

// initialize shift register to all blocked
blockAllSources();
}

// Convert timer ticks to seconds (original calculation: 1MHz/8 -> each tick is 125ns)
float getCurrentTimeSeconds(void) {
    unsigned long totalTicks = (overflowTicks * 65536UL) + TA0R;
    return (float)(totalTicks * 8.0) / 1000000.0; // seconds
}

// ===== MAIN =====
int main(void) {
    setup();

```

```

while (1) {
    // ——— Evaluation phase: for each source, charge evaluation capacitor
    unsigned int i;
    for (i = 0; i < MAX_SOURCES; ++i) {
        // Ensure all sources blocked
        blockAllSources();

        // Unblock and enable only this source
        setSourceSwitches(i, true, false); // SWS on, SWB off

        // Turn on evaluation capacitor switch to charge it from the selected source
        P2OUT |= CSW;
        __delay_cycles(500000); // 0.5s to charge (same as original)

        // Read cap high and cap low
        analogReadToIndex(ADC_CAP_HIGH_CH, 2); __bis_SR_register(CPUOFF);
        analogReadToIndex(ADC_CAP_LOW_CH, 3);  __bis_SR_register(CPUOFF);

        int ch = (int)adc_results[2];
        int cl = (int)adc_results[3];
        evalDiffs[i] = ch - cl;

        // disconnect evaluation cap and unblock the current source
        P2OUT &= ~CSW;
        setSourceSwitches(i, false, true); // SWS off, SWB on (block)

        __delay_cycles(1000000); // discharge wait 1s (same as original)
    }

    // Select primary = source with maximum evalDiff
    int primary = 0; int maxVal = evalDiffs[0];
    for (i = 1; i < MAX_SOURCES; ++i) {
        if (evalDiffs[i] > maxVal) { maxVal = evalDiffs[i]; primary = i; }
    }

    // configure switches for chosen primary: primary ON, others blocked
    for (i = 0; i < MAX_SOURCES; ++i) {
        if (i == (unsigned)primary) setSourceSwitches(i, true, false);
    }
}

```

```

        else setSourceSwitches(i, false, true); // block
    }

    // shift storage capacitor switch from evaluation to storage (same
P2OUT &= ~CSW;
P2OUT |= CSWS;

    // start time
    startTime = (unsigned long)getCurrentTimeSeconds();

    // Harvesting loop for HARVEST_SECONDS seconds
    totalTime = 0.0f;
    while (totalTime <= (float)HARVEST_SECONDS) {
        // sample reference once per loop
        analogReadToIndex(ADC_REF_CH, 4); __bis_SR_register(CPUOFF + G
        int vref = (int)adc_results[4];

        // read primary voltage
        unsigned int vp_raw = readMuxedSource(primary);
        int vp = (int)vp_raw;
        int vp_ref = vp - vref;

        // iterate through all other sources and add/block based on pha
        unsigned int s;
        for (s = 0; s < MAX_SOURCES; ++s) {
            if (s == (unsigned)primary) continue;

            unsigned int vs_raw = readMuxedSource(s);
            int vs = (int)vs_raw;
            int vdiff = vs - vp; // similar to original x10

            // If primary is in the same polarity as (secondary - prima
            if (((vp_ref > 0) && (vdiff > 0)) || ((vp_ref < 0) && (vdi
                // add secondary
                setSourceSwitches(s, true, false); // sws on, swb off
            } else {
                // block secondary
                setSourceSwitches(s, false, true); // sws off, swb on

```

```

        }
    }

    __delay_cycles(5000);
    endTime = (unsigned long)getCurrentTimeSeconds();
    totalTime = endTime - startTime;
}

// Reset timer counters after harvest
if (totalTime > (float)HARVEST_SECONDS) {
    overflowTicks = 0;
    TA0R = 0;
    startTime = 0;
    endTime = 0;
    totalTime = 0;
}
}

return 0; // never reached
}

```


Bibliography

- [1] P. D. Mitcheson, E. M. Yeatman, G. K. Rao, A. S. Holmes and T. C. Green, "Energy harvesting from human and machine motion for wireless electronic devices," *Proceedings of the IEEE*, vol. 96, no. 9, pp. 1457–1486, Sep. 2008.
- [2] J. A. Paradiso and T. Starner, "Energy scavenging for mobile and wireless electronics," *IEEE Pervasive Computing*, vol. 4, no. 1, pp. 18–27, Jan.–Mar. 2005.
- [3] Y. Tadesse, H. Rajamani and G. Misra, "Multisource energy harvesting for low-power wireless applications," *IEEE Transactions on Power Electronics*, vol. 30, no. 5, pp. 2761–2772, May 2015.
- [4] Texas Instruments, "*MSP430x2xx Family User's Guide*," Literature Number: SLAU144, Revised July 2022.
- [5] Analog Devices, "*LTC3388 – 20V Nanopower Synchronous Buck Converter Datasheet*," Document No. 4741, 2021.
- [6] Microchip Technology, "*TC7660 Charge Pump Voltage Converter*," DS21469B, 2020.
- [7] ON Semiconductor, "*CD4051B Analog Multiplexer/Demultiplexer Datasheet*," Rev. 6, July 2021.
- [8] Nexperia, "*74HC595 – 8-bit Serial-In/Serial-Out Shift Register*," Product Data Sheet, 2019.
- [9] The MathWorks Inc., "*Simulink Documentation – Signal Processing and Power Systems*,"
- [10] A. Raj, "*Design and Implementation of Intelligent Dual-Source Energy Harvesting Architecture Using MSP430 Microcontroller*," M.Tech Thesis, Advanced Technology Development Centre (ATDC), Indian Institute of Technology Kharagpur, 2024.